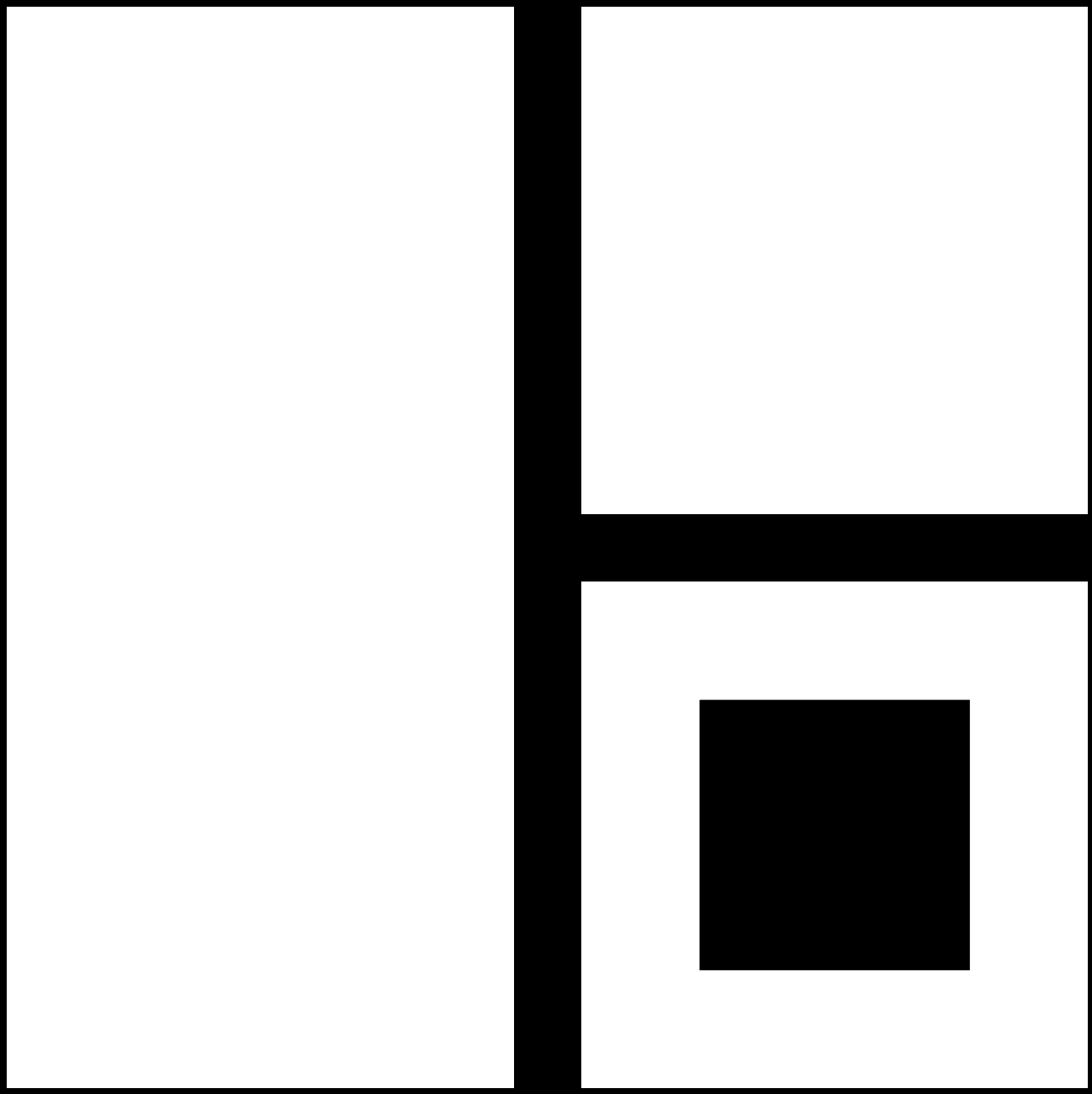


Bento

Manual técnico

Instalação, uso de cada módulo, personalização da interface em Tailwind CSS, API, permissões, tempo real, relatórios e integrações do painel administrativo.

Versão 1.0.0 Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

1 **Apresentação** 3

1.1 O que vem no projeto 3
1.2 Tecnologias 4
1.3 Estrutura de pastas 4
1.4 Como ler este manual 5

2 **Instalação** 6

2.1 Requisitos 6
2.2 Com Docker 6
2.3 Sem Docker 7
2.4 Primeiro acesso 7
2.5 Ambiente de desenvolvimento 8
2.6 Começando com o banco vazio 8
2.7 Em produção 9

3 **Usando o painel** 10

3.1 Menu, cabeçalho e busca 10
3.2 Visão geral 10
3.3 Analytics 11
3.4 Usuários 12
3.5 Notificações 13
3.6 Relatórios 14
3.7 Configurações 15
3.8 Componentes 16
3.9 No celular 17

4 **Interface em Tailwind CSS** 19

4.1 Como o CSS é organizado 19
4.2 Tokens de cor 19
4.3 Modo escuro 20
4.4 Paletas 20
4.5 Grade bento 21
4.6 Variantes personalizadas 22
4.7 Criando componentes 22

5 **JavaScript da interface** 24

5.1 Organização 24
5.2 Ciclo de uma página 24
5.3 Falando com a API 25
5.4 Tabela de dados 25
5.5 Gráficos 25
5.6 Gavetas, diálogos e avisos 26
5.7 Criando uma página nova 26

Copyright (©) 2026 Erik Pimenta

6 **API** 27

6.1 Convenções 27
6.2 Autenticação e sessões 27
6.3 Rotas de sessão, visão geral e analytics 28
6.4 Rotas de usuários e transações 28
6.5 Rotas de notificações e relatórios 29
6.6 Rotas de configurações e integrações 30
6.7 Exemplos 31
6.8 Adicionando uma rota 31

7 **Permissões** 32

7.1 Papéis 32
7.2 Matriz de permissões 32
7.3 Exceções por pessoa 33
7.4 Criando uma permissão 33

8 **Tempo real** 34

8.1 Eventos 34
8.2 Várias instâncias 34
8.3 Reconexão e indicador 34
8.4 Simulador de atividade 35

9 **Relatórios** 36

9.1 Tipos e colunas 36
9.2 Formatos 36
9.3 Envios agendados 36
9.4 Criando um tipo de relatório 37

10 **Integrações** 38

10.1 Provedores e campos 38
10.2 Eventos enviados 38
10.3 Webhook assinado 38
10.4 Segredos protegidos 39

11 **Configuração** 40

11.1 Variáveis de ambiente 40
11.2 Dados da empresa 40
11.3 Segurança aplicada 41

12 **Testes, build e CI** 42

12.1 Comandos 42
12.2 O que os testes cobrem 42
12.3 Integração contínua 42

13 **Solução de problemas** 43

13.1 Instalação e inicialização 43
13.2 Dados e acesso 43
13.3 Tempo real e integrações 44

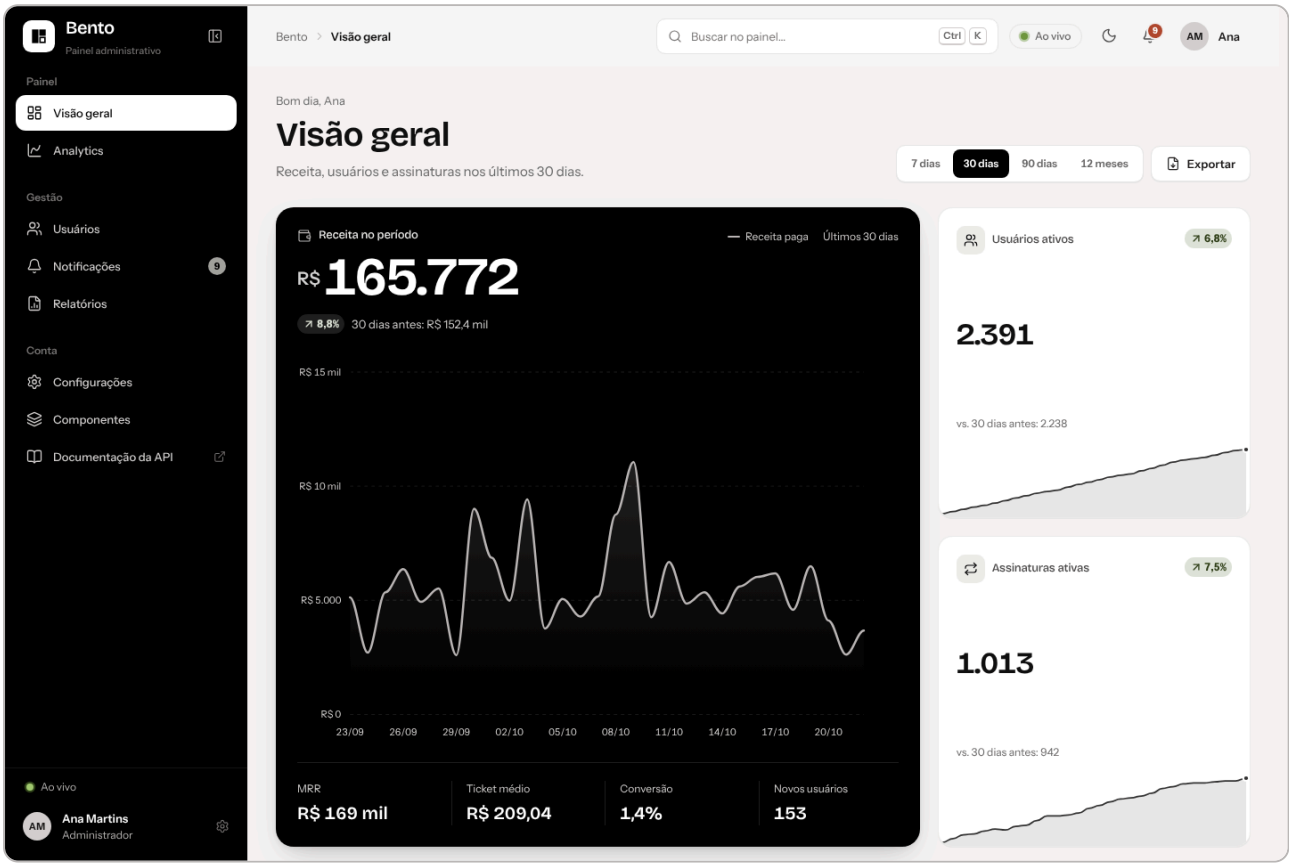
14 **Licenças** 45

14.1 Componentes de terceiros 45
14.2 Licença do Bento 45

Copyright (©) 2026 Erik Pimenta

1 Apresentação

O Bento é um painel administrativo para produtos SaaS. Este manual mostra como instalar, usar, adaptar e estender cada parte do projeto, da interface em Tailwind CSS à API e ao banco de dados.



Visão geral com a receita do período em destaque, KPIs com tendência e mini gráficos.

1.1 O que vem no projeto

O repositório traz uma aplicação completa, pronta para rodar com um comando e organizada para ser lida, copiada e modificada. As partes conversam por uma API REST documentada, então você pode trocar qualquer uma delas sem reescrever as outras.

- **Interface** com oito páginas: visão geral, analytics, usuários, notificações, relatórios, configurações, componentes e entrada. Modo escuro, 16 paletas, densidade compacta e menu recolhível.
- **Biblioteca de componentes em Tailwind CSS** com mais de 40 grupos: botões, formulários, tabelas, gavetas, kanban, calendário, fatura, preços, páginas de erro e modelos de grade bento.
- **API REST** em TypeScript com autenticação por sessão, permissões por papel, notificações em tempo real, relatórios em CSV e PDF, agendador, integrações e

Copyright (©) 2026 Erik Pimenta

documentação interativa.

- **Banco PostgreSQL** com migrações versionadas e dados de demonstração. Sem PostgreSQL instalado, a API usa o PGlite, o mesmo PostgreSQL embutido no processo.
- **Scripts de inicialização** para Windows, Linux e macOS, imagem Docker, Docker Compose, testes automatizados e integração contínua no GitHub Actions.

1.2 Tecnologias

Camada	Tecnologia	Função
Estilos	Tailwind CSS 4	Tokens de tema, variantes, componentes e utilitários escritos direto no CSS
Interface	HTML5 e JavaScript (módulos ES)	Páginas e controladores sem framework, fáceis de ler e portar
Gráficos	ApexCharts 4.7.0	Áreas, barras, rosca, radial, mapa de calor e mini gráficos
Ícones	Lucide	Ícones em SVG embutidos sob demanda
Fontes	Instrument Sans, Bricolage Grotesque, IBM Plex Mono	Texto, títulos e números monoespaçados
Build	Vite 7	Servidor de desenvolvimento com recarga e build multipágina
API	Node.js 22, TypeScript 5.9, Fastify 5	Servidor HTTP, rotas, plugins e documentação OpenAPI
Validação	Zod 4	Esquemas de entrada e saída de cada rota
Banco	PostgreSQL 18 ou PGlite 0.5	Mesmo SQL nos dois: servidor externo ou embutido
Arquivos	PDFKit	Relatórios em PDF paginados
Infraestrutura	Docker Compose, GitHub Actions	Execução com um comando e verificação a cada envio

1.3 Estrutura de pastas

O projeto usa workspaces do npm: a pasta `web` guarda a interface e a pasta `api` guarda o servidor. Os comandos da raiz chamam os dois.

```
SaaS-Admin-Dashboard/
  web/                      interface
    *.html                  uma página por módulo
    src/css/                Tailwind CSS
      tema/                 cores, paletas, tipografia, medidas e variantes
      base/                 documento, tipografia, formulários e impressão
      layout/               casca, grade bento, página e autenticação
      componentes/          32 arquivos de componentes do painel
      kit/                  30 arquivos do kit de interface
      paginas/              ajustes de cada página
    src/js/
      nucleo/               API, sessão, tema, tempo real, formatação e ícones
      componentes/          tabela, gráficos, gavetas, diálogos, avisos e abas
      layout/               menu, cabeçalho, busca global e início das páginas
```

Copyright (©) 2026 Erik Pimenta

```
    paginas/              controlador de cada página
  public/                 favicon, logo e script de tema
  tests/                  testes do núcleo
api/                      API em TypeScript
  src/modulos/            auth, visão geral, analytics, usuários, transações,
                          notificações, relatórios, configurações, integrações
  src/banco/              conexão, migrações, semente e administrador inicial
  src/relatorios/         geração de CSV e PDF
  src/tempo-real/         barramento de eventos
  migracoes/              esquema SQL
  test/                   testes da API
scripts/                  servidor de desenvolvimento e .env no Windows
docs/                     manuais em PDF e imagens
iniciar.cmd, iniciar.sh   inicialização com um comando
Dockerfile, docker-compose.yml
```

1.4 Como ler este manual

Os capítulos seguem a ordem em que você vai precisar deles. Para colocar o painel no ar, leia os capítulos 2 e 11. Para conhecer as telas, o capítulo 3. Para mudar o visual ou criar páginas, os capítulos 4 e 5. Para integrar outro sistema, os capítulos 6 a 10. O capítulo 13 reúne os problemas mais comuns e como resolvê-los.

A arquitetura do banco de dados tem um documento próprio, **Banco-de-Dados-Bento.pdf**, com o diagrama de relacionamentos e todas as tabelas, campos, índices e restrições.

Copyright (©) 2026 Erik Pimenta

2 Instalação

O Bento roda de duas formas: com Docker, que sobe o painel, a API e o PostgreSQL juntos, ou só com o Node.js, usando o banco embutido. As duas usam os mesmos scripts.

2.1 Requisitos

Para rodar com	Você precisa de	Observação
Docker	Docker com Docker Compose v2	Sobe o PostgreSQL 18 em um contêiner e guarda os dados em um volume
Node.js	Node.js 22 ou mais novo, com npm	Usa o PGlite: nada mais para instalar
Navegador	Chrome, Edge, Firefox ou Safari atuais	O painel usa diálogos nativos, módulos ES e Server-Sent Events

Por padrão o painel usa a porta `8080`. Em desenvolvimento, o Vite usa também a `5173`.

2.2 Com Docker

1. Baixe o projeto pelo botão **Code › Download ZIP** do GitHub e descompacte, ou clone com `git clone https://github.com/ravelroot/SaaS-Admin-Dashboard.git`.
2. No Windows, dê dois cliques em `iniciar.cmd`. No Linux ou no macOS, abra um terminal na pasta e rode `./iniciar.sh`.
3. Na primeira vez, o script cria o arquivo `.env` a partir do `.env.example`, com uma senha aleatória para o banco e um segredo aleatório para as sessões.
4. O script roda `docker compose up -d --build`, espera a rota `/api/v1/saude` responder e abre o navegador em **http://localhost:8080**.

Comandos úteis depois da primeira vez:

```
docker compose up -d --build      subir ou atualizar depois de mudar o código
docker compose logs -f bento      acompanhar os logs da API
docker compose ps                 ver se os contêineres estão saudáveis
docker compose down               parar, mantendo os dados
docker compose down -v            parar e apagar o banco (volta aos dados de demonstração)
```

> **Onde ficam os dados**
>
> O PostgreSQL guarda tudo no volume `bento_dados-postgres`. Ele sobrevive a `docker compose down` e a atualizações da imagem; só `down -v` apaga.

Copyright (©) 2026 Erik Pimenta

2.3 Sem Docker

Se o Docker não estiver instalado ou não estiver rodando, os mesmos scripts usam o Node.js: instalam as dependências, constroem a interface e a API e iniciam o servidor. Para fazer isso à mão:

```
git clone https://github.com/ravelroot/SaaS-Admin-Dashboard.git
cd SaaS-Admin-Dashboard
npm install
npm run build
npm start
```

Sem a variável `DATABASE_URL`, a API usa o PGlite e guarda o banco na pasta `dados/pglite`. Na primeira vez ela cria as tabelas e os dados de demonstração em poucos segundos. Se `BENTO_SEGREDO_JWT` não estiver definido, um segredo aleatório é gerado e salvo em `dados/segredo`.

Para usar um PostgreSQL que você já tem, crie um banco vazio e defina a conexão antes de iniciar:

```
DATABASE_URL=postgres://usuario:senha@servidor:5432/bento npm start
```

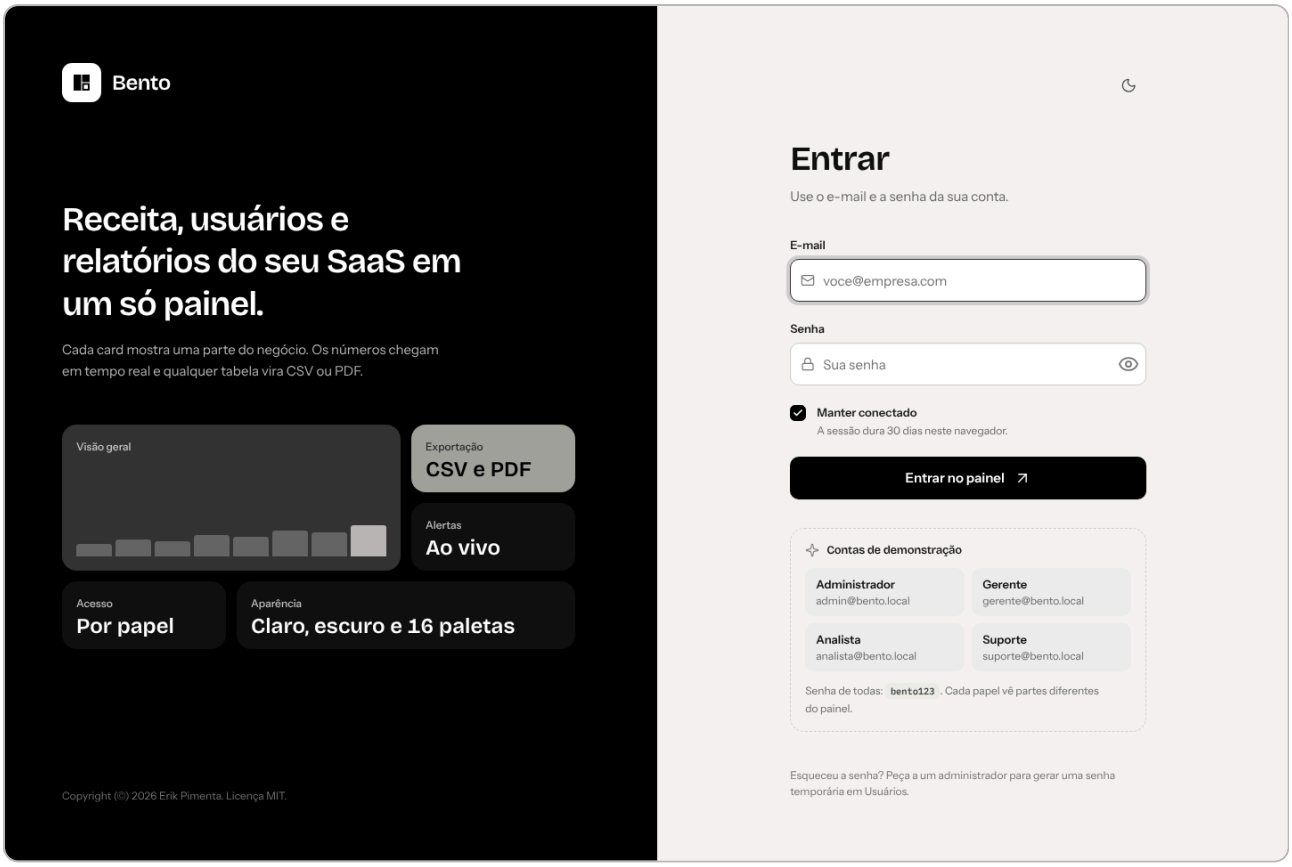
As migrações rodam sozinhas a cada inicialização. Para rodá-las sem subir o servidor, use `npm run migrar`.

2.4 Primeiro acesso

Abra **http://localhost:8080** e entre com uma das contas de demonstração. Todas usam a senha `bento123`.

E-mail	Papel	O que vê
`admin@bento.local`	Administrador	Tudo, inclusive dados da empresa e permissões
`gerente@bento.local`	Gerente	Tudo, menos dados da empresa, exclusão de usuários e permissões
`analista@bento.local`	Analista	Visão geral, analytics, transações, usuários (só leitura) e relatórios
`suporte@bento.local`	Suporte	Visão geral, transações e usuários (ver, editar e bloquear)

Copyright (©) 2026 Erik Pimenta



Tela de entrada com as contas de demonstração.

> **Antes de publicar**
> As contas de demonstração existem só para conhecer o painel. Em um servidor público, comece com o banco vazio (seção 2.6) ou bloqueie e exclua essas contas em **Usuários**.

2.5 Ambiente de desenvolvimento

```
npm run dev
```

O comando sobe a API na porta 8080 com recarga automática quando o TypeScript muda, e o Vite em **http://localhost:5173**. O Vite recarrega a página a cada mudança no Tailwind CSS, no HTML ou no JavaScript e encaminha tudo que começa com `/api` para a API.

Abra o endereço do Vite durante o desenvolvimento. O endereço da porta 8080 serve a versão construída por `npm run build`.

2.6 Começando com o banco vazio

Para usar o Bento com os seus próprios dados, desligue a semente e o simulador e peça à API para criar o primeiro administrador. No `.env`:

```
BENTO_SEMEAR=nao
BENTO_SIMULADOR=false
BENTO_ADMIN_EMAIL=voce@suaempresa.com
BENTO_ADMIN_SENHA=uma-senha-com-8-ou-mais-caracteres
BENTO_ADMIN_NOME=Seu Nome
```

Na inicialização, se ainda não existir uma conta com esse e-mail, a API cria um administrador ativo e registra o evento nas atividades. Se o e-mail já existir, nada muda. Depois de entrar, cadastre a equipe em **Usuários** e apague as três variáveis `BENTO_ADMIN_*`.

Em desenvolvimento, `npm run admin` cria a mesma conta sem subir o servidor. Com Docker, as variáveis passam do `.env` para o contêiner automaticamente.

> **Já usou com dados de demonstração?**
> A semente só roda em banco vazio. Para recomeçar, apague a pasta `dados/` (PGlite) ou rode `docker compose down -v` (PostgreSQL) antes de mudar o `.env`.

2.7 Em produção

A imagem Docker já roda a versão construída, com usuário sem privilégios e verificação de saúde. Antes de abrir o painel na internet, confira:

- **Segredo próprio.** Defina `BENTO_SEGREDO_JWT` com pelo menos 32 caracteres aleatórios. Ele assina as sessões e cifra os segredos das integrações; trocá-lo encerra todas as sessões e exige salvar as integrações de novo.
- **HTTPS.** Coloque o Bento atrás de um proxy reverso com certificado (Caddy, Nginx, Traefik ou o balanceador da sua nuvem) e defina `BENTO_HTTPS=true` para cookies seguros e HSTS.
- **Proxy confiável.** Com proxy reverso, defina `BENTO_PROXY=true` para que limites de requisição, sessões e atividades registrem o IP real.
- **Sem demonstração.** Use `BENTO_SEMEAR=nao`, `BENTO_SIMULADOR=false` e o administrador inicial da seção anterior.
- **Backups.** Programe um `pg_dump` diário (veja o capítulo Operação do documento do banco de dados). Com PGlite, copie a pasta `dados/` com o servidor parado.

Exemplo de proxy reverso com Caddy, que obtém o certificado sozinho:

```
painel.suaempresa.com {
    reverse_proxy localhost:8080 {
        flush_interval -1
    }
}
```

O `flush_interval -1` entrega os eventos em tempo real sem atraso. No Nginx, o equivalente é `proxy_buffering off` na rota `/api/v1/notificacoes/fluxo`.

Copyright (©) 2026 Erik Pimenta

3 Usando o painel

Cada página do Bento cuida de uma parte do negócio. Este capítulo passa por todas, na ordem em que aparecem no menu.

3.1 Menu, cabeçalho e busca

O menu lateral agrupa as páginas em **Painel**, **Gestão** e **Conta**. Em telas grandes, o botão ao lado da marca recolhe o menu para só ícones, e a escolha fica salva no navegador. No celular, o menu vira uma gaveta aberta pelo botão do cabeçalho. Itens que a pessoa não tem permissão para ver não aparecem.

O cabeçalho mostra a trilha de navegação, a busca global, o indicador de conexão em tempo real, a troca de tema claro e escuro, a central de notificações com o número de não lidas e o menu da conta.

Busca global

A busca encontra usuários por nome ou e-mail e transações pelo código ou pelo cliente, além de levar a qualquer página e às ações rápidas: cadastrar usuário, gerar relatório, alternar o tema, ajustar a aparência e sair.

Atalho	O que faz
`Ctrl` `K` ou `⌘` `K`	Abre a busca de qualquer página
`/`	Abre a busca quando nenhum campo está em edição
`↑` `↓`	Percorre os resultados
`Enter`	Abre o resultado selecionado
`Esc`	Fecha a busca, gavetas e diálogos

3.2 Visão geral

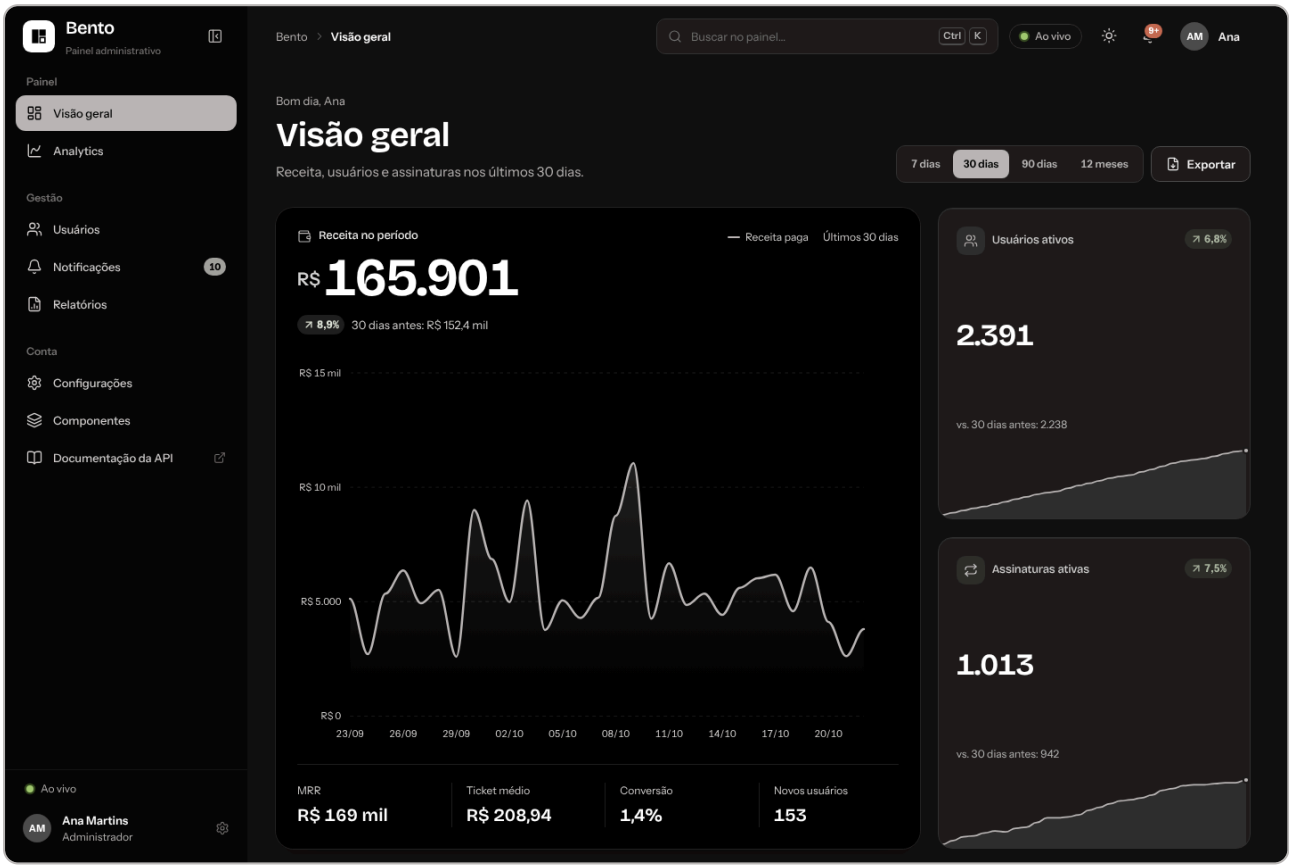
A página inicial resume o negócio no período escolhido no seletor do topo: 7 dias, 30 dias, 90 dias ou 12 meses. O cartão principal mostra a receita paga com o gráfico do período e, embaixo, MRR, ticket médio, conversão e usuários novos.

- **KPIs com tendência:** usuários ativos, assinaturas ativas, crescimento do MRR, cancelamentos e outros. A seta e o percentual comparam com o período anterior de mesmo tamanho; verde é melhora e vermelho é piora (em cancelamentos, a lógica se inverte).
- **Receita mês a mês** nos últimos 12 meses, com reembolsos e a linha da meta.
- **Assinaturas por plano**, **tráfego por canal** e o medidor da **meta do mês**.

Copyright (©) 2026 Erik Pimenta

- **Transações recentes** e **atividades** da equipe, atualizadas ao vivo. Cada transação abre uma gaveta com os detalhes e a assinatura ligada a ela.

O botão **Exportar** leva à página de relatórios, para gerar o arquivo do período que você quiser.



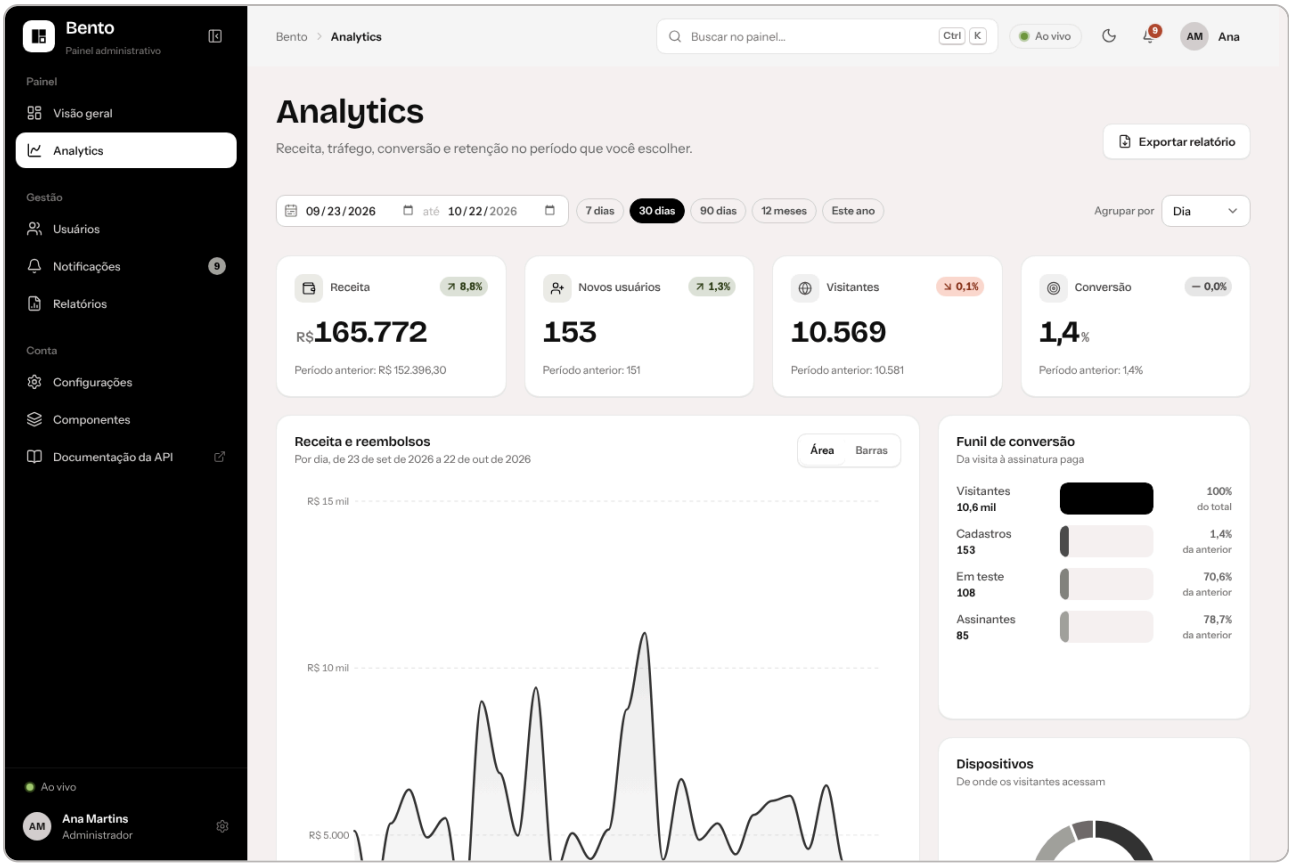
A mesma página no tema escuro.

3.3 Analytics

Analytics aprofunda os números com filtros de intervalo de datas, períodos prontos e agrupamento por dia, semana ou mês. Os filtros ficam na URL, então um link copiado abre a mesma análise.

- Resumo com receita, transações, usuários novos, visitantes, conversão, ticket médio, cancelamentos e MRR, cada um com a variação.
- Receita e reembolsos, usuários novos e ativos, assinaturas novas e cancelamentos ao longo do tempo.
- Funil de conversão (visitantes, cadastros, em teste, assinantes), dispositivos, canais com taxa de conversão e os oito países com mais visitantes.
- Receita por plano, mapa de calor das compras por dia da semana e hora, e retenção mensal por coorte dos últimos seis meses.

Copyright (©) 2026 Erik Pimenta



Analytics com filtros de período, funil e dispositivos.

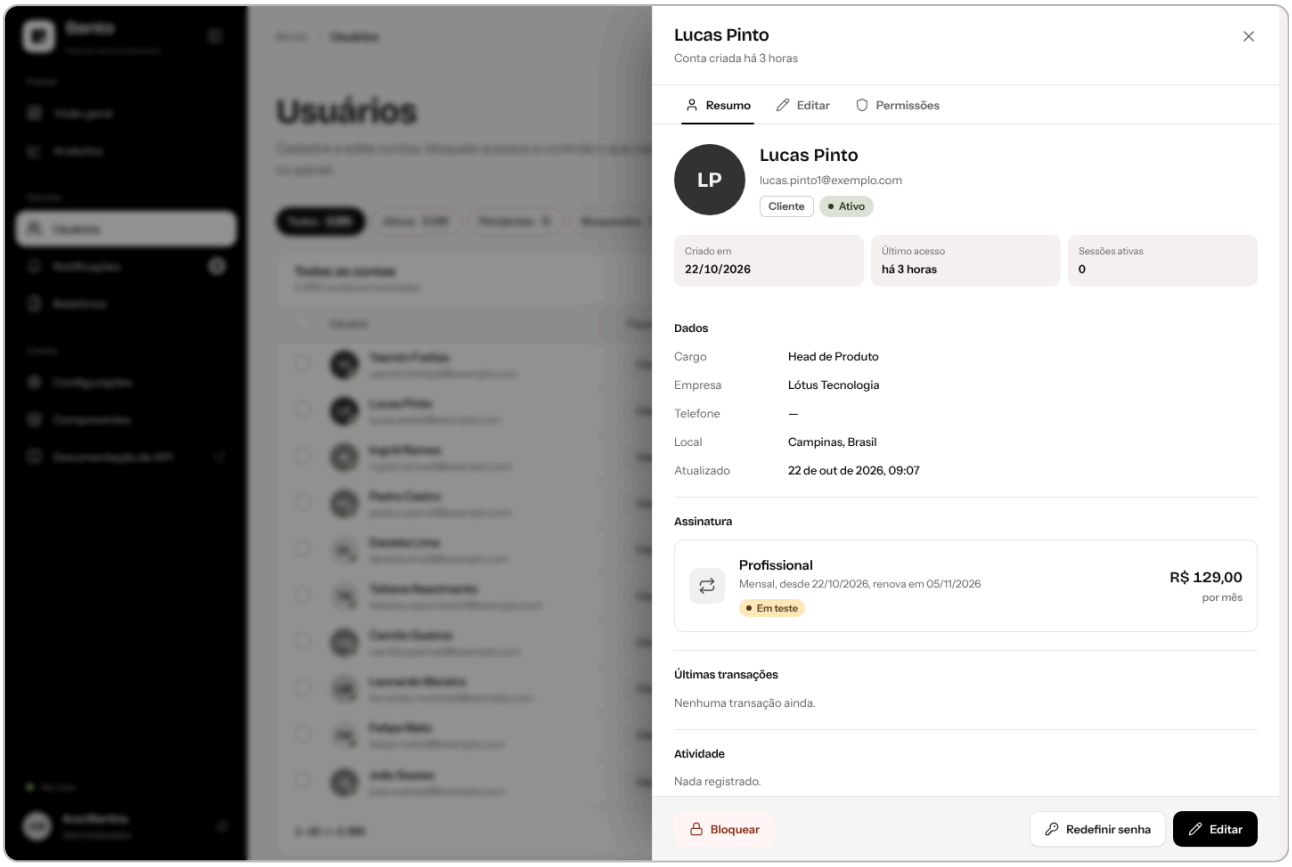
3.4 Usuários

A lista de usuários tem busca instantânea por nome e e-mail, filtros por status, papel e plano, contagens por status, colunas ordenáveis e paginação. As escolhas ficam na URL.

- **Cadastrar:** o botão **Novo usuário** abre uma gaveta com nome, e-mail, papel, cargo, empresa, telefone, país e cidade. Sem senha informada, o Bento gera uma senha temporária de 12 caracteres e mostra uma única vez para você copiar.
- **Detalhes:** clicar em uma linha abre a gaveta com as abas **Resumo** (dados, assinatura, últimas transações, atividades e sessões ativas), **Editar** e **Permissões**.
- **Bloquear e desbloquear:** o bloqueio pede um motivo, encerra as sessões da pessoa na hora e gera uma notificação de segurança.
- **Redefinir senha:** gera uma nova senha temporária.
- **Ações em massa:** selecione várias linhas para bloquear, desbloquear, ativar ou excluir de uma vez.
- **Exportar:** baixa a lista filtrada em CSV.

Na aba **Permissões**, quem tem a permissão de controlar permissões muda o papel da pessoa e marca exceções: permissões concedidas além do papel ou negadas mesmo que o papel tenha. O capítulo 7 explica as regras.

Copyright (©) 2026 Erik Pimenta



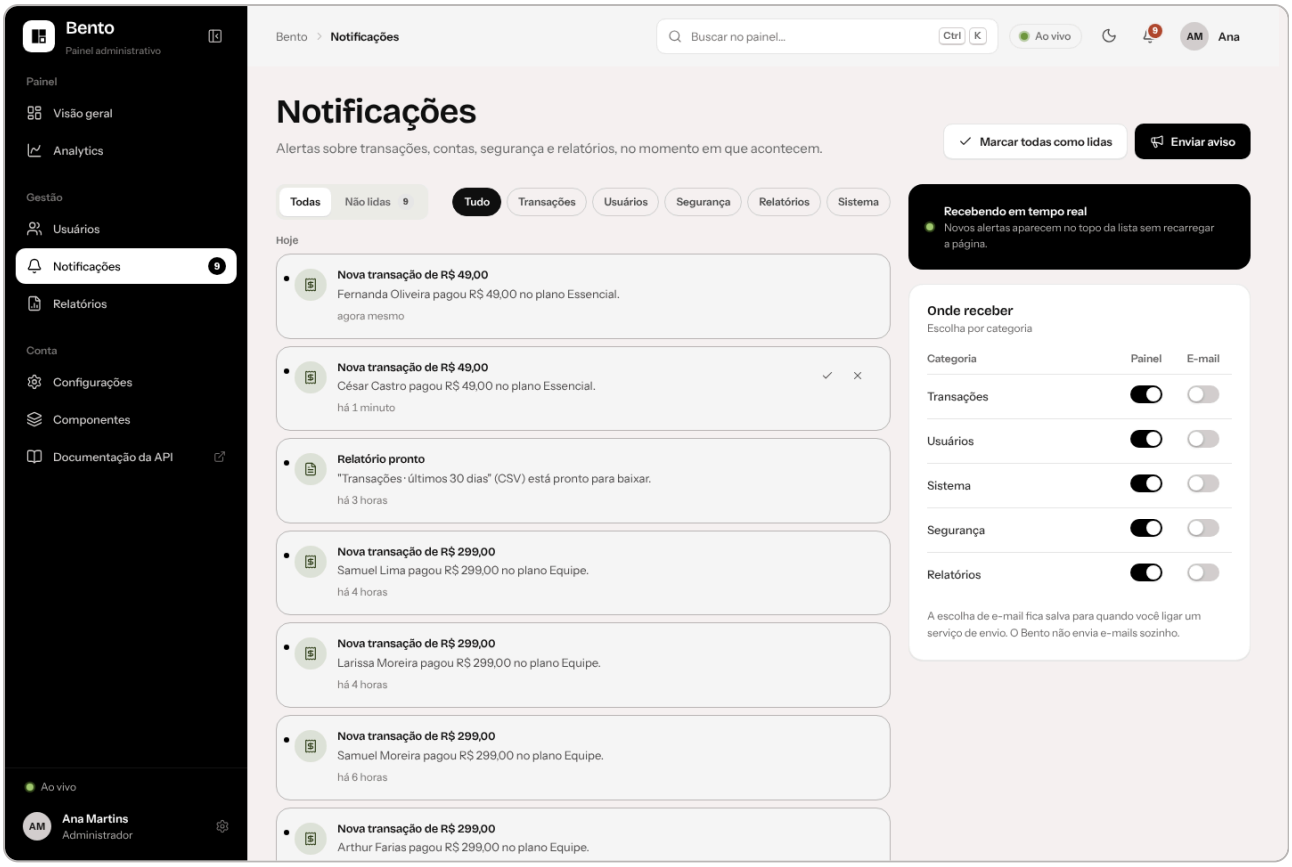
Lista de usuários com a gaveta de detalhes aberta.

3.5 Notificações

Notificações chegam em tempo real pelo cabeçalho e ficam guardadas nesta página, com filtros por categoria (transações, usuários, sistema, segurança e relatórios) e por lidas ou não lidas. Você pode marcar uma ou todas como lidas e dispensar as que não interessam; dispensar vale só para você.

Em **Preferências**, cada pessoa escolhe quais categorias quer receber. Quem tem a permissão de enviar avisos publica mensagens para toda a equipe ou para uma pessoa, com tipo informativo, sucesso, alerta ou erro.

Copyright (©) 2026 Erik Pimenta



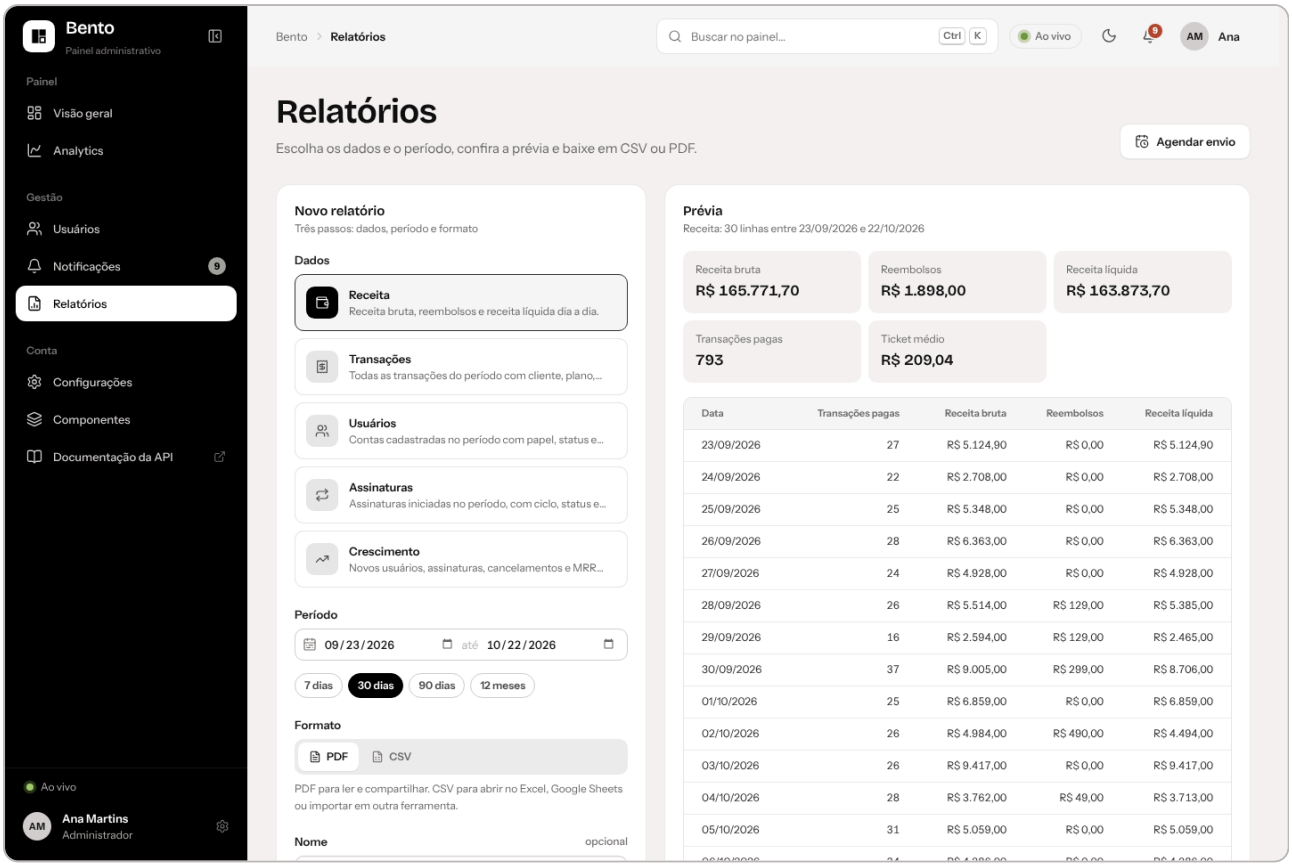
Notificações com filtros, preferências e envio de avisos.

3.6 Relatórios

Escolha o tipo (receita, transações, usuários, assinaturas ou crescimento), o período e o formato. A prévia mostra as primeiras linhas e o resumo antes de gerar. O arquivo pronto entra no histórico, de onde pode ser baixado de novo por qualquer pessoa com acesso aos relatórios.

Os **envios agendados** geram o mesmo relatório todo dia, toda semana ou todo mês, sempre às 7h no fuso configurado, e avisam quem criou o agendamento. O capítulo 9 detalha as colunas e os formatos.

Copyright (©) 2026 Erik Pimenta

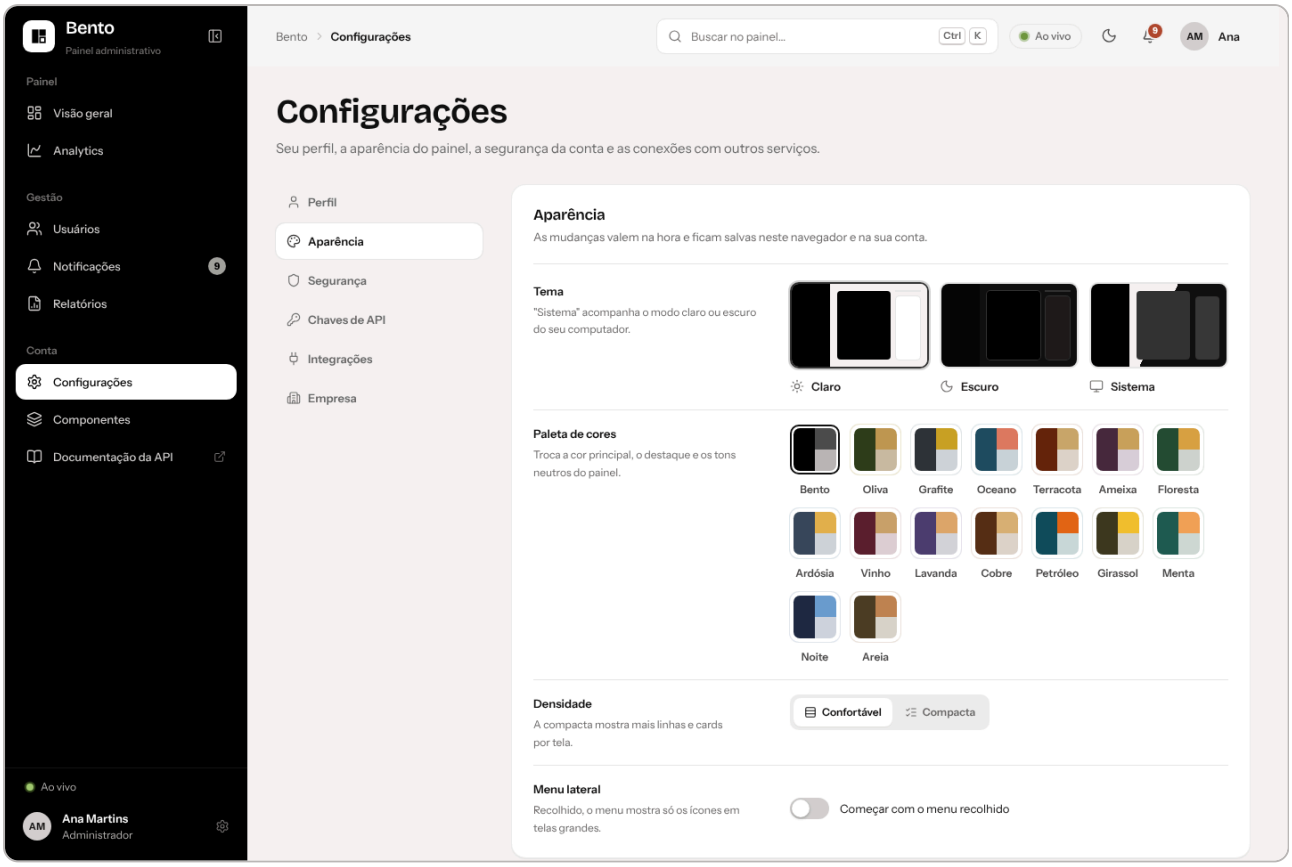


Relatórios com prévia, histórico e envios agendados.

3.7 Configurações

Aba	O que tem
Perfil	Nome, e-mail (pede a senha atual para trocar), cargo, telefone, empresa, país e cidade
Aparência	Tema claro, escuro ou do sistema, 16 paletas, densidade confortável ou compacta e menu recolhido. Salva no navegador e na conta, então vale em qualquer computador
Segurança	Troca de senha (encerra as outras sessões) e lista de sessões ativas com navegador, sistema, IP e último uso, com opção de encerrar uma ou todas
Chaves de API	Criação de chaves com escopo de leitura ou de escrita. O segredo aparece uma vez; o Bento guarda só o hash
Integrações	Webhook, Slack, Discord, Google Analytics e Stripe, com teste de conexão
Empresa	Nome da empresa, moeda, fuso e meta mensal de receita (só administradores)

Copyright (©) 2026 Erik Pimenta



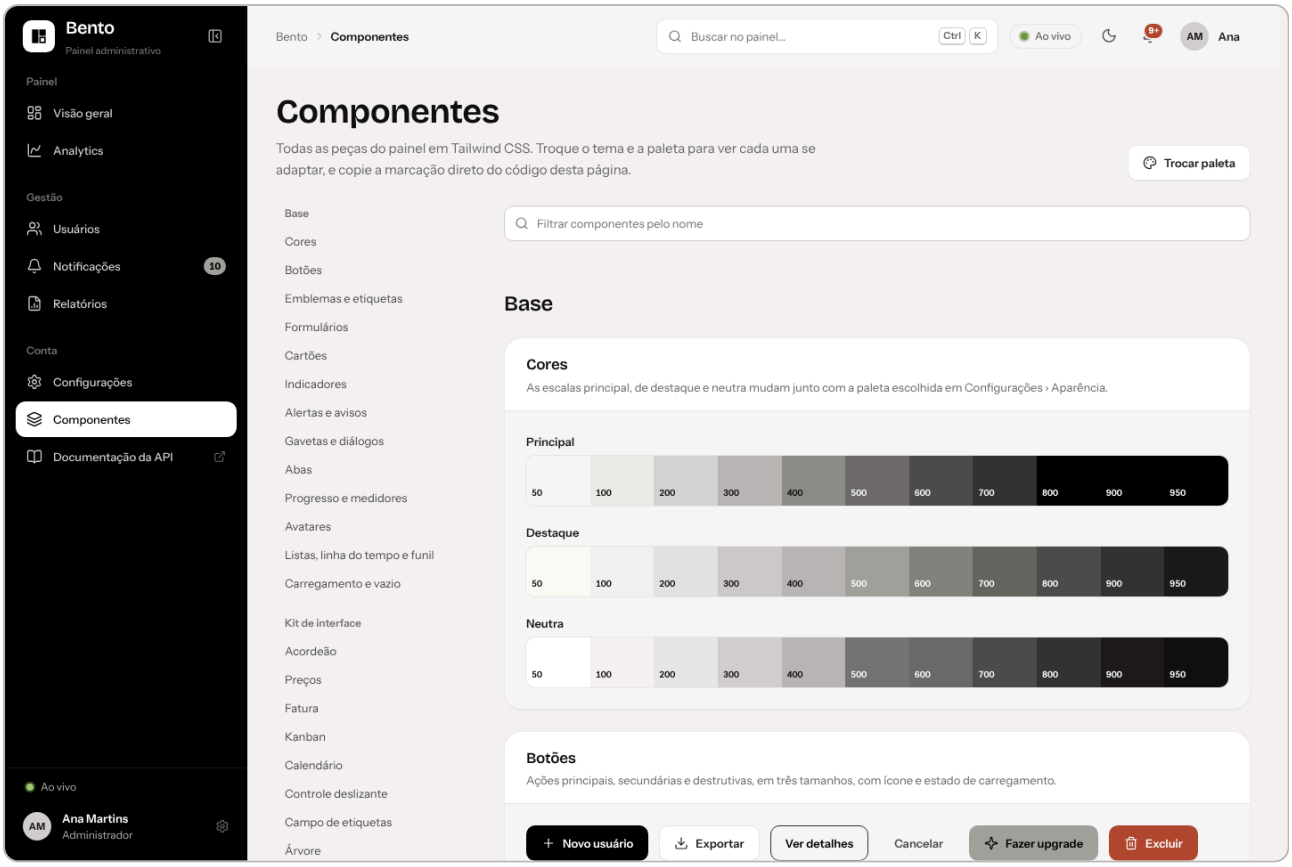
Aparência com tema, paletas e densidade.

3.8 Componentes

A página Componentes mostra toda a biblioteca em Tailwind CSS com exemplos funcionando: base e cores, botões, emblemas, formulários, cartões, indicadores, alertas, gavetas e diálogos, abas, progresso, avatares, listas e linha do tempo, carregamento e estados vazios, e o kit de interface com acordeão, preços, fatura, kanban, calendário, controle deslizante, campo de etiquetas, árvore, lista de arquivos, comentários, passo a passo, página de status, registro de mudanças, cartões de perfil, gráficos em CSS, filtros e seleção avançados, banners, páginas de erro, boas-vindas, pagamento, uso do plano, equipe, sobreposições, relatório para impressão e modelos de grade bento.

Use a página como catálogo: copie a marcação do exemplo, cole na sua página e ajuste o conteúdo. As classes funcionam em qualquer paleta e nos dois temas.

Copyright (©) 2026 Erik Pimenta

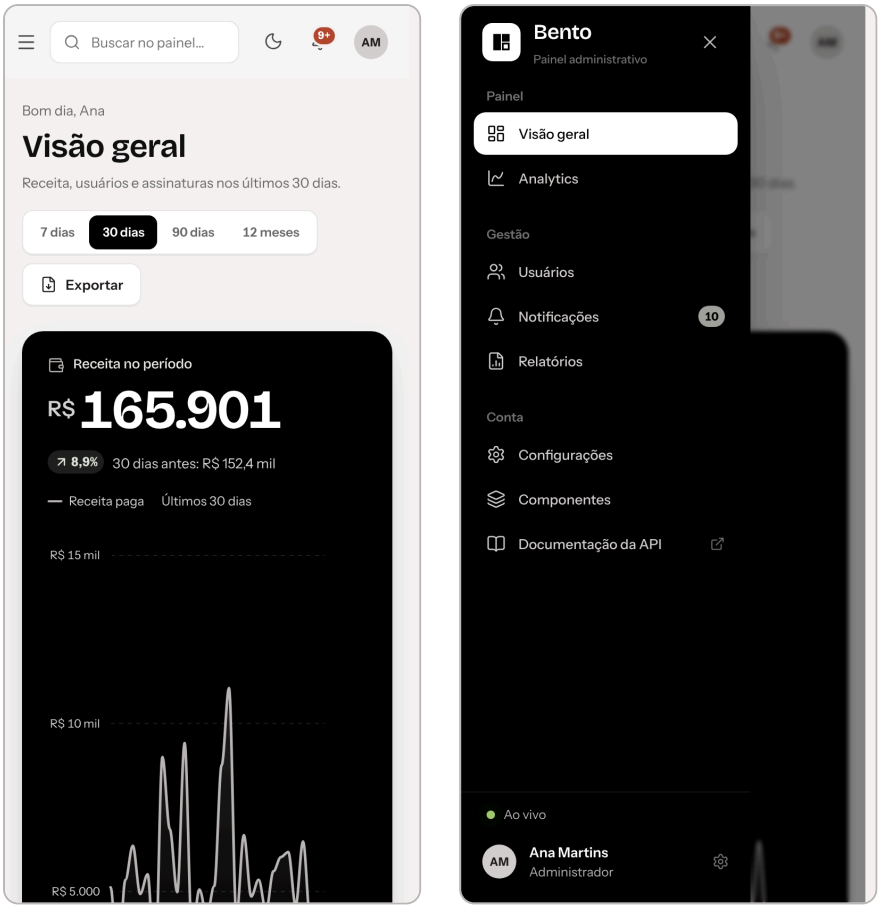


Biblioteca de componentes.

3.9 No celular

Todas as páginas se reorganizam no tablet e no celular: a grade bento passa de 12 para 6 colunas e depois para uma, as tabelas rolam na horizontal dentro do próprio cartão, as gavetas ocupam a tela inteira e o menu vira uma gaveta lateral.

Copyright (©) 2026 Erik Pimenta



Visão geral e menu no celular.

Copyright (©) 2026 Erik Pimenta

4 Interface em Tailwind CSS

Todo o visual do Bento está em Tailwind CSS 4, escrito direto nos arquivos .css. Este capítulo mostra onde fica cada coisa e como mudar cores, grade e componentes sem quebrar o resto.

4.1 Como o CSS é organizado

O ponto de entrada é `web/src/css/bento.css`. Ele importa o Tailwind e depois os arquivos do projeto, sempre nesta ordem, para que cada camada possa usar a anterior:

Pasta	Conteúdo	Quando mexer
`tema/`	Variantes personalizadas, escalas de cor, paletas, fontes, tamanhos, raios, sombras e animações	Para mudar a identidade visual
`base/`	Estilos do documento, tipografia, campos de formulário e regras de impressão	Raramente
`layout/`	Casca (menu e cabeçalho), grade bento, cabeçalho de página e tela de entrada	Para mudar a estrutura das páginas
`componentes/`	Os 32 componentes usados pelo painel	Para ajustar um componente existente
`kit/`	Os 30 grupos do kit de interface e os temas dos gráficos	Para adicionar componentes novos
`paginas/`	Ajustes que valem para uma página só	Para detalhes de uma página
`utilitarios/`	Utilitários extras criados com @utility	Para atalhos reutilizáveis

Os componentes ficam em `@layer components` e são montados com `@apply`, então continuam sendo classes do Tailwind: dá para combinar `class="botao botao-primario w-full"` e qualquer utilitário vence o componente quando necessário.

4.2 Tokens de cor

As cores são variáveis CSS declaradas com `@theme` em `tema/cores.css`, o que gera as classes do Tailwind automaticamente (`bg-primaria-800`, `text-neutra-600`, `border-destaque-400` e assim por diante). A paleta padrão, Bento, é monocromática:

Cor	Hex	Papel na interface
Branco	`#FFFFFF`	Fundo dos cartões e texto sobre preto
Cinza claro	`#BAB7B6`	Bordas, divisores, trilhas de gráfico e texto de apoio no escuro
Chumbo	`#4D4B4B`	Texto secundário, ícones e séries de gráfico
Grafite	`#333232`	Superfícies do tema escuro e estados de foco

Copyright (©) 2026 Erik Pimenta

Cor	Hex	Papel na interface
Preto	`#000000`	Texto principal, menu lateral, botões primários e cartão de destaque

Essas cores formam três escalas de 50 a 950: `primaria` (ações e destaques), `destaque` (acentos e segunda série dos gráficos) e `neutra` (fundos, bordas e texto). Além delas existem `creme` e `tinta`, o claro e o escuro absolutos de cada paleta.

Cores de status

Quatro escalas fixas comunicam estado e não mudam com a paleta: `oliva` para sucesso (paga, ativa, conectada), `ouro` para alerta (pendente, em teste), `brasa` para erro (falhou, bloqueado, inadimplente) e `anil` para informação (reembolsada). As setas de tendência dos KPIs usam verde e vermelho próprios para continuar legíveis sobre o preto.

4.3 Modo escuro

O modo escuro usa a variante `dark:` ligada à classe `.dark` no elemento `<html>`, declarada em `tema/variantes.css`:

```
@custom-variant dark (&:where(.dark, .dark *));
```

O script `web/public/tema-inicial.js` roda antes de a página aparecer, lê a escolha guardada no `localStorage` (chaves com o prefixo `bento:`) e aplica a classe, sem piscar o tema errado. Com a opção **Sistema**, o painel segue a preferência do sistema operacional e muda junto se ela mudar.

Ao escrever um componente novo, declare as duas versões lado a lado:

```
.meu-cartao {
  @apply rounded-cartao border border-neutra-200 bg-neutra-50 text-neutra-900
         dark:border-neutra-800 dark:bg-neutra-900 dark:text-neutra-100;
}
```

4.4 Paletas

Uma paleta é um bloco que redefine as escalas `primaria`, `destaque` e `neutra` quando o `<html>` tem o atributo `data-paleta`. Como todos os componentes usam essas escalas, trocar a paleta muda o painel inteiro, inclusive os gráficos, sem recarregar.

Paleta	Cor principal	Paleta	Cor principal
Bento (padrão)	#000000	Vinho	#5C1F2E
Oliva	#31401A	Lavanda	#4B3F72
Grafite	#2E3338	Cobre	#592D14
Oceano	#1F4E5F	Petróleo	#0F4C5C

Copyright (©) 2026 Erik Pimenta

Paleta	Cor principal	Paleta	Cor principal
Terracota	#65240E	Girassol	#3D3A1E
Ameixa	#4A2C40	Menta	#1E5E52
Floresta	#234F33	Noite	#1F2A44
Ardósia	#3B4A5A	Areia	#4B3D26

Criando uma paleta

1. Em `web/src/css/tema/paletas.css`, copie um bloco existente e troque o nome e os 33 valores (11 tons para cada escala). Mantenha o tom 50 quase branco e o 950 quase preto, para o contraste dos dois temas.
2. No mesmo arquivo, adicione as três regras de `.paleta-amostra` com as cores que representam a paleta na tela de aparência.
3. Acrescente o nome à lista `PALETAS` em `web/src/js/nucleo/tema.js` e a opção em `web/configuracoes.html`.
4. Na API, inclua o nome na validação de `api/src/modulos/configuracoes/rotas.ts` e `api/src/modulos/comum/consultas.ts`, e crie uma migração nova que atualize a restrição `preferencias_usuario_paleta_check`.

```
[data-paleta="minha"] {
  --color-primaria-50: #F6F7FB;
  --color-primaria-500: #5B6A9A;
  --color-primaria-800: #24315C;
  --color-primaria-950: #0B1024;
}
```

O exemplo mostra só quatro tons por brevidade; declare os onze de cada escala.

4.5 Grade bento

A grade fica em `layout/bento.css`. O contêiner `.bento` tem 1 coluna no celular, 6 a partir de 768 px e 12 a partir de 1280 px, com linhas de altura mínima fixa e preenchimento denso, para que cartões de tamanhos diferentes se encaixem sem buracos.

Classe	Tablet (6 col.)	Desktop (12 col.)
`.bento-quarto`	3	3
`.bento-terco`	3	4
`.bento-cinco-doze`	6	5
`.bento-metade`	6	6
`.bento-sete-doze`	6	7
`.bento-dois-tercos`	6	8

Copyright (©) 2026 Erik Pimenta

Classe	Tablet (6 col.)	Desktop (12 col.)
`.bento-tres-quartos`	6	9
`.bento-inteiro`	6	12
`.bento-alto` / `.bento-muito-alto`	1 linha	2 / 3 linhas
`.bento-pilha`	2 cartões lado a lado	4 colunas, cartões empilhados
`.bento-kpis`	2 por linha	4 por linha

```
<div class="bento">
  <article class="cartao cartao-destaque bento-dois-tercos bento-alto">...</article>
  <div class="bento-pilha">
    <article class="cartao">...</article>
    <article class="cartao">...</article>
  </div>
  <article class="cartao bento-terco">...</article>
  <article class="cartao bento-terco">...</article>
  <article class="cartao bento-terco">...</article>
</div>
```

A seção **Modelos de grade bento** da página Componentes traz arranjos prontos para vendas, produto, suporte, financeiro e marketing.

4.6 Variantes personalizadas

Além de `dark:`, o Bento declara variantes que reagem ao estado da interface. Elas funcionam como qualquer variante do Tailwind:

Variante	Ativa quando	Exemplo
`recolhida:`	O menu lateral está recolhido	`recolhida:hidden`
`compacta:`	A densidade compacta está ligada	`compacta:py-2`
`aberta:`	Um elemento com data-aberta="true" envolve o atual	`aberta:rotate-180`
`ativo:`	Item com aria-current, aria-selected ou data-ativo	`ativo:bg-neutra-200`
`ordenado:`	Cabeçalho de tabela com aria-sort	`ordenado:text-tinta`
`movimento:`	A pessoa não pediu movimento reduzido	`movimento:animate-surgir`

As animações respeitam `prefers-reduced-motion`, e todos os elementos interativos têm foco visível pelo teclado.

4.7 Criando componentes

1. Crie um arquivo em `web/src/css/kit/`, por exemplo `meu-componente.css`, com as classes dentro de `@layer components`.

Copyright (©) 2026 Erik Pimenta

2. Use só as escalas do tema (`primaria` , `destaque` , `neutra` , `creme` , `tinta` e as de status), nunca cores fixas, para o componente funcionar em todas as paletas e nos dois temas.
3. Importe o arquivo em `web/src/css/bento.css` , junto dos outros do kit.
4. Adicione um exemplo em `web/componentes.html` , para que o componente fique documentado no catálogo.

> **Limite do @apply**
>
> No Tailwind 4, `@apply` aceita utilitários e classes criadas com `@utility` , mas não outras classes de componente. Para reaproveitar um conjunto de utilitários em vários componentes, crie um `@utility` , como `selecao-filtro` em `componentes/filtros.css` .

Copyright (©) 2026 Erik Pimenta

5 JavaScript da interface

A interface usa JavaScript puro em módulos ES, sem framework. Cada página tem um controlador pequeno que monta a casca, busca os dados na API e liga os componentes.

5.1 Organização

Pasta	Arquivos	Responsabilidade
`nucleo/`	api, sessao, tema, tempo-real, dom, formato, icones, eventos, armazenamento	Comunicação com a API, sessão e permissões, tema, eventos ao vivo, marcação segura, formatação em pt-BR e ícones
`componentes/`	tabela, graficos, gaveta, modal, aviso, abas, menu-suspenso, formulario, indicadores, transacao	Peças reutilizáveis de interface
`layout/`	inicio, casca, navegacao, busca-global	Início de cada página, menu, cabeçalho e busca
`paginas/`	Um arquivo por página	Regras de cada tela

5.2 Ciclo de uma página

Toda página chama `iniciarPagina`, de `layout/inicio.js`. A função carrega a sessão (e manda para a tela de entrada se ela tiver expirado), aplica as preferências de aparência da conta, monta o menu e o cabeçalho, abre a conexão em tempo real, confere a permissão da página e só então chama o seu código.

```
import { iniciarPagina } from "../layout/inicio.js";
import { api } from "../nucleo/api.js";
import { html, renderizar } from "../nucleo/dom.js";

iniciarPagina({
  pagina: "estoque",
  permissao: "painel.ver",
  aoIniciar: async ({ conteudo }) => {
    const dados = await api.obter("/estoque");
    renderizar(conteudo.querySelector("[data-lista]"),
      html`<ul>${dados.itens.map((item) => html`<li>${item.nome}</li>`)}</ul>`);
  },
});
```

O `html` é uma template tag que escapa todo valor interpolado, o que protege contra injeção de HTML. Para inserir marcação confiável de propósito, use `seguro()`.

Copyright (©) 2026 Erik Pimenta

5.3 Falando com a API

O objeto `api` de `nucleo/api.js` cobre os métodos HTTP e cuida dos detalhes: prefixo `/api/v1`, JSON, cookies de sessão, o cabeçalho `X-Bento: 1` exigido nas alterações e a renovação automática da sessão quando o acesso expira.

Função	Método HTTP
`api.obter(caminho, consulta)`	GET com parâmetros de consulta
`api.criar(caminho, corpo)`	POST
`api.atualizar(caminho, corpo)`	PATCH
`api.substituir(caminho, corpo)`	PUT
`api.excluir(caminho)`	DELETE
`api.baixar(caminho, consulta)`	GET que salva o arquivo com o nome enviado pela API

Erros chegam como `ErroApi`, com `status`, `codigo` e `detalhes`. A função `mensagemDeErro(erro)` devolve um texto pronto para mostrar ao usuário.

5.4 Tabela de dados

A classe `TabelaDados` monta tabelas com ordenação, busca com espera de digitação, filtros, paginação, seleção de linhas e estados de carregamento e vazio. Ela chama uma função sua para buscar cada página, então funciona com qualquer rota paginada da API.

```
const tabela = new TabelaDados(elemento, {
  colunas: [
    { campo: "nome", titulo: "Nome", ordenavel: true, render: (item) => item.nome },
    { campo: "criadoEm", titulo: "Cadastro", ordenavel: true, render: (item) =>
dataCurta(item.criadoEm) },
  ],
  ordenacao: { campo: "criadoEm", direcao: "desc" },
  selecionavel: true,
  buscar: (parametros, sinal) => api.obter("/usuarios", parametros, { sinal }),
});
```

Os `parametros` trazem `pagina`, `porPagina`, `ordenar`, `direcao`, `q` e os filtros, nos mesmos nomes que a API espera. O `sinal` cancela a busca anterior quando a pessoa continua digitando.

5.5 Gráficos

A classe `Grafico` envolve o ApexCharts: recebe o elemento, uma função que monta as opções a partir dos dados e das cores do tema, e redesenha sozinha quando o tema ou a paleta mudam.

Copyright (©) 2026 Erik Pimenta

```
const grafico = new Grafico(elemento, (dados, cores) => ({
  chart: { type: "area", height: 260 },
  series: [{ name: "Receita", data: dados.valores }],
  xaxis: { categories: dados.rotulos },
  yaxis: { labels: { formatter: eixoMoeda } },
  colors: [cores.primaria],
}));
await grafico.renderizar(dados);
```

Para gráficos pequenos sem biblioteca, `faisca()` desenha a linha de tendência dos KPIs em SVG, `barrasMini()` desenha barras e `medidor()` desenha o anel da meta.

5.6 Gavetas, diálogos e avisos

- `new Gaveta({ titulo, subtitulo, largura })` cria uma gaveta lateral com o elemento `<dialog>` nativo, com foco preso dentro dela e fechamento por `Esc`.
- `confirmar({ titulo, texto, tipo, confirmar })` pede confirmação e devolve uma promessa; com `tipo: "perigo"`, o botão fica no estilo de ação destrutiva.
- `mostrarSegredo({ titulo, texto, segredo })` mostra uma senha temporária ou chave de API com botão de copiar.
- `avisar({ tipo, titulo, texto })`, `avisoSucesso()` e `avisoErro()` mostram avisos temporários no canto da tela.

5.7 Criando uma página nova

1. Copie um arquivo HTML da pasta `web` (por exemplo `notificacoes.html`), troque o título, o conteúdo de `#conteudo` e o script do fim para `/src/js/paginas/estoque.js`.
2. Adicione o nome ao vetor `paginas` de `web/vite.config.js`, para o build incluir a página.
3. Adicione a rota em `PAGINAS` de `api/src/estatico.ts`, para a API servir a página construída em `/estoque`.
4. Inclua o item em `NAVEGACAO` de `web/src/js/layout/navegacao.js`, com ícone, rótulo, endereço e permissão.
5. Crie o controlador em `web/src/js/paginas/` chamando `iniciarPagina`, e os estilos específicos em `web/src/css/paginas/`, importados em `bento.css`.

Copyright (©) 2026 Erik Pimenta

6 API

Tudo que o painel faz passa por uma API REST em /api/v1. Ela pode ser usada por outros sistemas com uma chave de API, e a documentação interativa fica em /api/v1/docs.

6.1 Convenções

- Todas as rotas começam com `/api/v1`. Corpos e respostas são JSON, com nomes de campos em português e camelCase.
- Valores em dinheiro vêm em centavos inteiros no campo `valor`, acompanhados de `moeda` (`"BRL"` por padrão). Datas e horas seguem a ISO 8601.
- Listas aceitam `pagina` (a partir de 1), `porPagina` (padrão 10, máximo 100), `ordenar`, `direcao` (`asc` ou `desc`) e `q` para busca, e respondem com `{ itens, total, pagina, porPagina, paginas }`.

Erros

Todo erro tem o mesmo formato, com uma mensagem pronta em português:

```
{ "erro": { "codigo": "VALIDACAO", "mensagem": "email: e-mail inválido", "detalhes": [ ... ] } }
```

Código	Status	Quando acontece
`NAO_AUTENTICADO`	401	Sem sessão ou com acesso expirado
`CREDENCIAIS`	401	E-mail ou senha incorretos
`SEM_PERMISSAO`	403	A conta não tem a permissão da rota
`SEM_ACESSO_PAINEL`	403	O papel da conta não acessa o painel (clientes)
`CSRF`	403	Alteração por cookie sem o cabeçalho X-Bento
`NAO_ENCONTRADO`	404	O registro não existe
`CONFLITO`	409	E-mail já cadastrado, ação sobre a própria conta e casos parecidos
`VALIDACAO`	422	Corpo ou parâmetros inválidos; detalhes traz cada campo
`BLOQUEADO`	423	A conta está bloqueada
`LIMITE`	429	Muitas requisições ou tentativas de login
`INTERNO`	500	Erro inesperado no servidor (registrado no log)

6.2 Autenticação e sessões

O login em `POST /auth/entrar` cria dois cookies httpOnly: `bento_sessao`, com um JWT de acesso válido por 15 minutos, e `bento_renovacao`, com um token aleatório restrito a `/api/v1/auth`. Quando o acesso expira, a interface chama `POST /auth/renovar`, que gira o token de renovação; reaproveitar um token antigo falha.

Copyright (©) 2026 Erik Pimenta

- Com **Lembrar de mim**, a sessão dura 30 dias; sem, 12 horas.
- Cada requisição confere a sessão no banco. Bloquear a conta, trocar a senha ou encerrar a sessão tem efeito imediato.
- Cinco senhas erradas seguidas suspendem o login da conta por 15 minutos e geram uma notificação de segurança.
- Pedidos que alteram dados com cookie precisam do cabeçalho `X-Bento: 1`. Outros sites não conseguem enviar esse cabeçalho, o que impede ataques de falsificação de requisição.

Chaves de API

Para integrar outro sistema, crie uma chave em **Configurações › Chaves de API** e envie no cabeçalho `Authorization: Bearer bnt_...`. A chave age com as permissões de quem a criou. O escopo `leitura` libera só pedidos GET; `escrita` libera também as alterações. Chaves dispensam o cabeçalho `X-Bento`.

Limites

Cada IP pode fazer 300 requisições por minuto. O login aceita 10 tentativas por minuto e a renovação, 60. Acima disso a API responde 429 com o código `LIMITE`.

6.3 Rotas de sessão, visão geral e analytics

Método	Rota	Permissão	O que faz
`GET`	`/saude`	`pública`	Status, tipo de banco, versão e tempo no ar
`POST`	`/auth/entrar`	`pública`	Login com email, senha e lembrar
`POST`	`/auth/sair`	sessão	Encerra a sessão e limpa os cookies
`POST`	`/auth/renovar`	sessão	Renova o acesso e gira o token
`GET`	`/auth/eu`	sessão	Conta, permissões, preferências e dados da empresa
`GET`	`/visao-geral?periodo=30d`	`painel.ver`	KPIs (7d, 30d, 90d ou 12m), receita mensal, planos, canais, meta, transações e atividades
`GET`	`/analytics?inicio&fim&agrupar`	`analytics.ver`	Resumo e séries por dia, semana ou mês, funil, canais, dispositivos, países, coortes e mapa de calor
`GET`	`/busca?q`	`painel.ver`	Até 5 usuários e 5 transações (mínimo 2 caracteres)
`GET`	`/atividades`	`auditoria.ver`	Registro de atividades, paginado, com filtro por usuário

6.4 Rotas de usuários e transações

Método	Rota	Permissão	O que faz
`GET`	`/usuarios`	`usuarios.ver`	Lista com q, status, papel, plano, ordenação e contagens por status

Copyright (©) 2026 Erik Pimenta

Método	Rota	Permissão	O que faz
POST	/usuarios	usuarios.criar	Cadastra; sem senha, devolve uma senha temporária uma vez
GET	/usuarios/{id}	usuarios.ver	Conta, assinatura, últimas transações e atividades, sessões ativas
PATCH	/usuarios/{id}	usuarios.editar	Edita dados; mudar o papel exige usuarios.permissoes
DELETE	/usuarios/{id}	usuarios.excluir	Exclui (não vale para a própria conta)
POST	/usuarios/{id}/bloquear	usuarios.bloquear	Bloqueia com motivo e encerra as sessões
POST	/usuarios/{id}/desbloquear	usuarios.bloquear	Desbloqueia
POST	/usuarios/{id}/redefinir-senha	usuarios.editar	Gera uma senha temporária
GET	/usuarios/{id}/permissoes	usuarios.ver	Papel, permissões do papel, concedidas, negadas e efetivas
PUT	/usuarios/{id}/permissoes	usuarios.permissoes	Define papel, concedidas e negadas
POST	/usuarios/acoes-em-massa	da ação	Bloquear, desbloquear, ativar ou excluir várias contas
GET	/usuarios/exportar	usuarios.ver	CSV com os mesmos filtros da lista
GET	/papeis	usuarios.ver	Papéis com permissões e número de usuários
GET	/permissoes	usuarios.ver	Catálogo de permissões
GET	/transacoes	transacoes.ver	Lista com q, status, método, tipo, datas e ordenação
GET	/transacoes/{id}	transacoes.ver	Detalhes com a assinatura ligada
GET	/transacoes/exportar	transacoes.ver	CSV com os mesmos filtros

6.5 Rotas de notificações e relatórios

Método	Rota	Permissão	O que faz
GET	/notificacoes	sessão	Lista (todas ou não lidas, por categoria) com o total não lido
GET	/notificacoes/contagem	sessão	Número de não lidas
GET	/notificacoes/fluxo	sessão	Eventos em tempo real (text/event-stream)
POST	/notificacoes/{id}/ler	sessão	Marca como lida
POST	/notificacoes/ler-todas	sessão	Marca todas como lidas
DELETE	/notificacoes/{id}	sessão	Dispensa só para quem pediu
POST	/notificacoes	notificacoes.enviar	Publica um aviso para todos ou para uma pessoa
GET	/notificacoes/preferencias	sessão	Categorias que a pessoa recebe
PUT	/notificacoes/preferencias	sessão	Salva as preferências
GET	/relatorios/tipos	relatorios.ver	Tipos com as colunas de cada um
GET	/relatorios/previa?tipo&inicio&fim	relatorios.ver	Até 25 linhas, total e resumo
GET	/relatorios	relatorios.ver	Histórico paginado
POST	/relatorios	relatorios.gerar	Gera em CSV ou PDF e guarda no histórico
GET	/relatorios/{id}/baixar	relatorios.ver	Baixa o arquivo

Copyright (©) 2026 Erik Pimenta

Método	Rota	Permissão	O que faz
DELETE	/relatorios/{id}	relatorios.gerar	Remove do histórico
GET	/relatorios/agendados	relatorios.ver	Envios agendados
POST	/relatorios/agendados	relatorios.agendar	Cria um agendamento
PATCH	/relatorios/agendados/{id}	relatorios.agendar	Pausa ou retoma
DELETE	/relatorios/agendados/{id}	relatorios.agendar	Remove

6.6 Rotas de configurações e integrações

Método	Rota	Permissão	O que faz
GET	/configuracoes/perfil	sessão	Dados da própria conta
PATCH	/configuracoes/perfil	sessão	Edita o perfil; trocar o e-mail pede a senha atual
GET	/configuracoes/aparencia	sessão	Tema, paleta, densidade e menu
PUT	/configuracoes/aparencia	sessão	Salva a aparência
POST	/configuracoes/seguranca/senha	sessão	Troca a senha e encerra as outras sessões
GET	/configuracoes/seguranca/sessoes	sessão	Sessões ativas
DELETE	/configuracoes/seguranca/sessoes/{id}	sessão	Encerra uma sessão
DELETE	/configuracoes/seguranca/sessoes	sessão	Encerra todas, menos a atual
GET	/configuracoes/chaves-api	sessão	Chaves da própria conta
POST	/configuracoes/chaves-api	sessão	Cria uma chave; o segredo aparece uma vez
DELETE	/configuracoes/chaves-api/{id}	sessão	Revoga
GET	/configuracoes/sistema	sessão	Nome da empresa, moeda, fuso, meta e idioma
PUT	/configuracoes/sistema	configuracoes.sistema	Salva os dados da empresa
GET	/integracoes	configuracoes.integracoes	Integrações com campos (segredos mascarados)
PATCH	/integracoes/{id}	configuracoes.integracoes	Ativa, desativa e configura
POST	/integracoes/{id}/testar	configuracoes.integracoes	Testa a conexão

Copyright (©) 2026 Erik Pimenta

6.7 Exemplos

Com cookie de sessão (curl)

```
curl -c sessao.txt -H "Content-Type: application/json" \
     -d '{"email":"admin@bento.local","senha":"bento123"}' \
     http://localhost:8080/api/v1/auth/entrar

curl -b sessao.txt "http://localhost:8080/api/v1/visao-geral?periodo=90d"

curl -b sessao.txt -H "X-Bento: 1" -H "Content-Type: application/json" \
     -d '{"motivo":"Pagamento contestado"}' \
     http://localhost:8080/api/v1/usuarios/ID_DO_USUARIO/bloquear
```

Com chave de API (JavaScript)

```
const resposta = await fetch("https://painel.suaempresa.com/api/v1/transacoes?
status=paga&porPagina=100", {
  headers: { Authorization: "Bearer " + process.env.BENTO_CHAVE },
});
const { itens, total } = await resposta.json();
```

Documentação interativa

Em **/api/v1/docs** a API publica todas as rotas com parâmetros, esquemas e exemplos, e permite testá-las no navegador com a sessão atual. A especificação OpenAPI em JSON fica em `/api/v1/docs/json`, pronta para gerar clientes em outras linguagens.

6.8 Adicionando uma rota

1. Crie a pasta do módulo em `api/src/modulos/` com um `rotas.ts` (e um `servico.ts` se a regra for grande).
2. Declare a rota com o esquema Zod de entrada, a etiqueta da documentação e o `preHandler` com `app.autenticar` e `exigir("permissao")`.
3. Registre a função em `api/src/app.ts`, junto das outras, dentro do prefixo `/api/v1`.
4. Escreva um teste em `api/test/`; o arquivo `auxiliar.ts` cria a API com um banco em memória já preenchido.

```
export async function rotasEstoque(app: FastifyInstance): Promise<void> {
  app.get('/estoque', {
    schema: { tags: ['Estoque'], summary: 'Itens em estoque', querystring: paginacaoSimples },
    preHandler: [app.autenticar, exigir('painel.ver')],
  }, async (requisicao) => {
    const { linhas } = await app.banco.consultar('select id, nome from estoque order by nome');
    return { itens: linhas };
  });
}
```

Copyright (©) 2026 Erik Pimenta

7 Permissões

O acesso no Bento é controlado por papéis com permissões, mais exceções por pessoa. A interface esconde o que a pessoa não pode usar, e a API confere tudo de novo em cada rota.

7.1 Papéis

Papel	Código	Para quem
Administrador	`admin`	Donos do sistema: acesso total, inclusive dados da empresa e permissões
Gerente	`gerente`	Gerencia usuários, relatórios e integrações, sem alterar o sistema
Analista	`analista`	Consulta métricas e analytics e gera relatórios
Suporte	`suporte`	Atende clientes: consulta, edita e bloqueia contas
Cliente	`cliente`	Contas da plataforma, sem acesso ao painel (o login responde 403)

7.2 Matriz de permissões

Permissão	O que libera	Adm.	Ger.	Anal.	Sup.
`painel.ver`	Acessa o painel e os indicadores principais	•	•	•	•
`analytics.ver`	Consulta gráficos, filtros por período e coortes	•	•	•	
`usuarios.ver`	Lista, pesquisa e abre o detalhe dos usuários	•	•	•	•
`usuarios.criar`	Cria contas novas	•	•		
`usuarios.editar`	Altera dados cadastrais e redefine senhas	•	•		•
`usuarios.bloquear`	Bloqueia e desbloqueia contas	•	•		•
`usuarios.excluir`	Remove contas definitivamente	•			
`usuarios.permissoes`	Muda o papel e as permissões individuais	•			
`transacoes.ver`	Consulta e exporta transações	•	•	•	•
`notificacoes.enviar`	Publica avisos para a equipe	•	•		
`relatorios.ver`	Consulta o histórico e baixa relatórios	•	•	•	
`relatorios.gerar`	Gera relatórios em CSV e PDF	•	•	•	
`relatorios.agendar`	Cria e pausa relatórios recorrentes	•	•	•	
`configuracoes.integracoes`	Conecta webhooks, Slack, Discord e outros serviços	•	•		
`configuracoes.sistema`	Altera nome da empresa, moeda, fuso e meta mensal	•			
`auditoria.ver`	Consulta o registro de atividades	•	•		

Clientes não têm nenhuma permissão. As regras estão em `api/migracoes/003_dados_base.sql`.

Copyright (©) 2026 Erik Pimenta

7.3 Exceções por pessoa

As permissões efetivas de uma pessoa são as do papel, mais as concedidas e menos as negadas na aba **Permissões** do usuário:

```
efetivas = permissões do papel  ∪  concedidas  −  negadas
```

Assim dá para deixar um analista exportar transações sem mudar o papel de todos os analistas, ou tirar a exclusão de um gerente específico. Algumas regras valem sempre: ninguém bloqueia nem exclui a própria conta, e só quem controla permissões cadastra administradores e gerentes.

Na interface, use `pode("relatorios.gerar")` de `nucleo/sessao.js` para mostrar ou esconder botões. Na API, `exigir("relatorios.gerar")` no `preHandler` da rota.

7.4 Criando uma permissão

1. Crie uma migração nova em `api/migracoes/`, por exemplo `004_estoque.sql`, que insira a permissão em `permissoes` e os papéis que a recebem em `papel_permissoes`.
2. Proteja as rotas com `exigir("estoque.ver")`.
3. Use `pode("estoque.ver")` na interface e a propriedade `permissao` no item do menu.

```
insert into permissoes (codigo, modulo, nome, descricao, ordem) values
  ('estoque.ver', 'Estoque', 'Ver estoque', 'Consulta os itens em estoque.', 90);

insert into papel_permissoes (papel, permissao) values
  ('admin', 'estoque.ver'), ('gerente', 'estoque.ver');
```

Copyright (©) 2026 Erik Pimenta

8 Tempo real

Transações, cadastros e notificações aparecem no painel no momento em que acontecem, sem recarregar a página. O Bento usa Server-Sent Events, que funcionam em HTTP comum e atravessam proxies sem configuração especial.

8.1 Eventos

Depois do login, a interface abre uma conexão em `GET /api/v1/notificacoes/fluxo`. A API envia `retry: 5000` logo no início, um comentário `: ping` a cada 25 segundos para manter a conexão aberta, e estes eventos:

Evento	Conteúdo	Quem usa
`notificacao`	A notificação nova, já filtrada para a pessoa	Central do cabeçalho e página de notificações
`contagem`	{ naoLidas }	Número no sino e no menu
`transacao`	A transação paga, com cliente e plano	Transações recentes da visão geral
`usuario`	O usuário recém-cadastrado	Atividades e lista de usuários

Para consumir o fluxo em outro sistema, basta um `EventSource` no navegador ou qualquer cliente SSE com o cookie de sessão.

8.2 Várias instâncias

Com PostgreSQL, cada evento é publicado com `pg_notify` no canal `bento_eventos`, e cada instância da API mantém uma conexão dedicada com `LISTEN` que repassa o evento aos navegadores conectados nela. Por isso você pode rodar várias instâncias atrás de um balanceador sem perder eventos. A conexão de escuta se refaz sozinha em 3 segundos se cair.

Com PGlite, que roda dentro de um único processo, os eventos passam por um emissor em memória com o mesmo contrato.

> **PgBouncer**
> Se usar PgBouncer em modo transação, aponte a conexão de escuta direto para o PostgreSQL: `LISTEN` não funciona por um pool de transação.

8.3 Reconexão e indicador

O indicador no cabeçalho mostra **Ao vivo**, **Conectando** ou **Reconectando**. Se a conexão cair, o navegador tenta de novo sozinho; ao voltar para a aba depois de um tempo, a interface reabre a conexão e atualiza os números.

Copyright (©) 2026 Erik Pimenta

8.4 Simulador de atividade

Com `BENTO_SIMULADOR=true` (padrão), a API cria transações de assinantes em intervalos aleatórios em torno de `BENTO_SIMULADOR_INTERVALO` segundos (padrão 25; o intervalo real varia entre 60% e 140% desse valor), às vezes pendentes ou recusadas, e um cadastro novo a cada cinco ciclos. Cada evento gera notificação, atividade e disparo para as integrações ativas, exatamente como um evento real.

O simulador existe para demonstrar o painel. Desligue em produção com `BENTO_SIMULADOR=false`.

Copyright (©) 2026 Erik Pimenta

9 Relatórios

Os relatórios transformam os dados do painel em arquivos para planilhas, contabilidade ou apresentação. Cada um tem colunas fixas, prévia antes de gerar e histórico para baixar de novo.

9.1 Tipos e colunas

Tipo	Uma linha por	Colunas
Receita	Dia do período	Data, transações pagas, receita bruta, reembolsos, receita líquida
Transações	Transação	Código, data, cliente, e-mail, plano, método, status, valor
Usuários	Usuário cadastrado no período	Nome, e-mail, papel, status, plano, país, cadastro
Assinaturas	Assinatura iniciada no período	Cliente, e-mail, plano, ciclo, status, valor, início, cancelamento
Crescimento	Mês	Mês, novos usuários, assinaturas novas, cancelamentos, MRR no fim do mês, crescimento do MRR

Cada relatório traz também um resumo (totais, médias e taxas) que aparece na prévia e no topo do PDF.

9.2 Formatos

- **CSV** em UTF-8 com BOM e ponto e vírgula como separador, para abrir direto no Excel em português. Valores em reais com vírgula decimal e datas no formato dd/mm/aaaa.
- **PDF** em A4 com a marca do Bento, título, período, data de geração, quadro de resumo e tabela com cabeçalho repetido a cada página e numeração no rodapé, nas cores da paleta monocromática.

O arquivo fica guardado no próprio banco, então qualquer instância da API consegue entregá-lo sem disco compartilhado. O nome segue o padrão `bento-receita-2026-09-22-a-2026-10-22.pdf`.

9.3 Envios agendados

Frequência	Quando roda	Períodos possíveis
Diária	Todo dia às 7h	Últimos 7 dias, últimos 30 dias ou mês anterior
Semanal	Toda segunda-feira às 7h	Os mesmos
Mensal	Todo dia 1º às 7h	Os mesmos

Copyright (©) 2026 Erik Pimenta

O horário segue `BENTO_FUSO`. Um agendador interno verifica a cada minuto os envios vencidos e reserva cada um com uma atualização condicional no banco, então só uma instância gera o arquivo mesmo com várias rodando. Quem criou o agendamento recebe uma notificação com o link do relatório pronto, e as integrações ativas recebem o evento `relatorio.gerado`.

9.4 Criando um tipo de relatório

1. Em `api/src/modulos/relatorios/dados.ts`, acrescente o tipo com nome, descrição e colunas (campo, título e formato: texto, moeda, número, data ou percentual).
2. Escreva a consulta que devolve as linhas e o resumo do período.
3. Crie uma migração que inclua o tipo novo nas restrições `relatorios_tipo_check` e `relatorios_agendados_tipo_check`.
4. A interface lê os tipos de `GET /relatorios/tipos`, então o novo tipo aparece sozinho na tela de relatórios.

Copyright (©) 2026 Erik Pimenta

10 Integrações

As integrações enviam os eventos do negócio para outros sistemas no momento em que acontecem: uma URL sua, um canal do Slack ou do Discord. Também guardam credenciais do Google Analytics e do Stripe para quem quiser estender a importação de dados.

10.1 Provedores e campos

Integração	Campos	O teste faz
Webhook	URL de destino (http ou https) e segredo de assinatura (12 a 200 caracteres)	Um POST real com evento de teste, assinado
Slack	URL do webhook de entrada (hooks.slack.com)	Uma mensagem real no canal
Discord	URL do webhook do canal (discord.com/api/webhooks)	Uma mensagem real no canal
Google Analytics	ID de medição (G-XXXXXXX) e segredo da API de medição	Valida o formato dos campos
Stripe	Chave publicável (pk_), chave secreta (sk_ ou rk_) e segredo do webhook (whsec_)	Valida o formato dos campos

Os envios têm limite de 5 segundos. Se um destino falhar, a integração fica com status de erro e a última mensagem aparece no cartão dela, sem atrapalhar o painel.

10.2 Eventos enviados

Evento	Quando
`transacao.paga`	Uma transação foi paga
`usuario.criado`	Uma conta nova foi criada, pelo painel ou por cadastro
`usuario.bloqueado`	Uma conta foi bloqueada
`relatorio.gerado`	Um relatório ficou pronto, gerado à mão ou por agendamento

10.3 Webhook assinado

O webhook recebe um POST com o corpo JSON abaixo e dois cabeçalhos: `X-Bento-Evento`, com o nome do evento, e `X-Bento-Assinatura`, com `sha256=` seguido do HMAC-SHA256 do corpo calculado com o seu segredo.

Copyright (©) 2026 Erik Pimenta

```
{
  "evento": "transacao.paga",
  "resumo": "Paula Lima pagou R$ 129,00 (Profissional)",
  "criadoEm": "2026-10-22T14:03:11.402Z",
  "dados": { "id": "...", "codigo": "TX-009214", "valor": 12900, "moeda": "BRL", ... }
}
```

Confira a assinatura antes de confiar no conteúdo. Em Node.js:

```
import { createHmac, timingSafeEqual } from "node:crypto";

function assinaturaValida(corpoBruto, cabecalho, segredo) {
  const esperado = "sha256=" + createHmac("sha256", segredo).update(corpoBruto).digest("hex");
  const a = Buffer.from(esperado);
  const b = Buffer.from(cabecalho ?? "");
  return a.length === b.length && timingSafeEqual(a, b);
}
```

Use o corpo exatamente como chegou, antes de convertê-lo em objeto, para que o HMAC bata.

10.4 Segredos protegidos

Campos do tipo segredo são cifrados com AES-256-GCM antes de ir para o banco, com uma chave derivada de `BENTO_SEGREDO_JWT`. A API nunca devolve o valor: a interface recebe só os quatro últimos caracteres, como `••••1234`. Ao editar uma integração, deixar um segredo em branco mantém o atual.

> **Troca do segredo da aplicação**
>
> Se você trocar `BENTO_SEGREDO_JWT`, os segredos já salvos não podem mais ser lidos. Abra cada integração ativa e informe os segredos de novo.

Copyright (©) 2026 Erik Pimenta

11 Configuração

Todo o comportamento do servidor é ajustado por variáveis de ambiente, lidas do arquivo .env na raiz do projeto ou do próprio ambiente. Nenhuma é obrigatória para rodar localmente.

11.1 Variáveis de ambiente

Variável	Padrão	O que faz
`BENTO_PORTA`	8080	Porta HTTP do servidor
`BENTO_HOST`	0.0.0.0	Interface de rede em que o servidor escuta
`DATABASE_URL`	vazio	Conexão do PostgreSQL. Vazio usa o PGlite embutido
`BENTO_DADOS`	dados/pglite	Pasta do PGlite; o valor memoria guarda tudo em memória
`BENTO_POOL`	10	Conexões no pool do PostgreSQL
`BENTO_SEGREDO_JWT`	gerado	Assina as sessões e cifra os segredos das integrações. Sem ele, é gerado e salvo em dados/segredo
`BENTO_HTTPS`	false	Cookies Secure, HSTS e atualização de requisições inseguras
`BENTO_ORIGENS`	vazio	Outras origens que podem chamar a API com credenciais, separadas por vírgula
`BENTO_PROXY`	false	Confia no X-Forwarded-For de um proxy reverso
`BENTO_SEMEAR`	auto	nao desliga os dados de demonstração em banco vazio
`BENTO_SIMULADOR`	true	Gera transações e cadastros de demonstração
`BENTO_SIMULADOR_INTERVALO`	25	Intervalo médio do simulador, em segundos
`BENTO_FUSO`	America/Sao_Paulo	Fuso das agregações por dia e do agendador
`BENTO_LOG`	info	Nível de log; debug mostra cada requisição
`BENTO_WEB`	web/dist	Pasta da interface construída que a API serve
`BENTO_ADMIN_EMAIL`	vazio	E-mail do administrador criado na inicialização, se ainda não existir
`BENTO_ADMIN_SENHA`	vazio	Senha desse administrador (mínimo 8 caracteres)
`BENTO_ADMIN_NOME`	Administrador	Nome desse administrador
`BENTO_DB_NOME`	bento	Banco criado pelo Docker Compose
`BENTO_DB_USUARIO`	bento	Usuário do banco no Docker Compose
`BENTO_DB_SENHA`	gerada	Senha do banco no Docker Compose, criada pelos scripts de inicialização

11.2 Dados da empresa

Nome da empresa, moeda, fuso e meta mensal ficam no banco e são editados em **Configurações › Empresa**. A meta mensal alimenta o medidor da visão geral e a linha de meta do gráfico de receita.

Copyright (©) 2026 Erik Pimenta

11.3 Segurança aplicada

- Senhas com bcrypt; tokens de sessão e chaves de API guardados só como hash SHA-256.
- Cabeçalhos de segurança com Helmet, incluindo política de conteúdo que só permite recursos da própria origem.
- Cookies httpOnly e SameSite=Lax, com Secure quando `BENTO_HTTPS=true`.
- Proteção contra falsificação de requisição pelo cabeçalho `X-Bento`, limites de requisição por IP e suspensão temporária após senhas erradas.
- Validação de toda entrada com Zod e consultas SQL sempre parametrizadas.
- Registro de atividades sem chave estrangeira obrigatória, para o histórico continuar mesmo depois de excluir uma conta.

Copyright (©) 2026 Erik Pimenta

12 Testes, build e CI

O projeto vem com testes automatizados para a API e para o núcleo da interface, checagem de tipos e um fluxo de integração contínua que sobe tudo contra um PostgreSQL de verdade.

12.1 Comandos

Comando	O que faz
`npm run dev`	API com recarga automática e Vite em http://localhost:5173
`npm run build`	Gera web/dist e api/dist
`npm start`	Sobe a versão construída em http://localhost:8080
`npm test`	Testes da API e do núcleo da interface
`npm run verificar`	Checagem de tipos do TypeScript sem gerar arquivos
`npm run migrar`	Aplica as migrações pendentes
`npm run semear`	Cria os dados de demonstração em um banco vazio
`npm run admin`	Cria o administrador definido em BENTO_ADMIN_*

12.2 O que os testes cobrem

- **API:** login e recusa de senha errada e de clientes, sessão obrigatória, CSRF, renovação girando o token, chave de API somente leitura, permissões do papel e individuais, listas com paginação, ordenação, filtros e validação, bloqueio com encerramento de sessões, visão geral de todos os períodos, analytics, prévia e geração de relatórios em CSV e PDF, agendamentos, notificações e o início com banco vazio e administrador inicial. Cada arquivo de teste usa um banco PGlite em memória próprio.
- **Interface:** escape de HTML na template tag, formatação de moeda, números, datas e tempos relativos em português.

Os testes usam o executor nativo do Node.js, sem dependências extras.

12.3 Integração contínua

O fluxo `.github/workflows/ci.yml` roda a cada envio e pull request, em três etapas:

- **Interface:** instala, roda os testes e constrói o Vite.
- **API:** checa os tipos, roda os testes, sobe um PostgreSQL 18 como serviço, inicia a API construída contra ele e faz requisições reais de login, visão geral e usuários.
- **Docker:** constrói a imagem de produção.

Copyright (©) 2026 Erik Pimenta

13 Solução de problemas

Os problemas abaixo são os mais comuns ao instalar e adaptar o Bento, com a causa e o que fazer.

13.1 Instalação e inicialização

A porta 8080 já está em uso

Outro programa usa a porta. Defina `BENTO_PORTA=8090` (ou outra livre) no `.env` e inicie de novo. Com Docker, a porta publicada muda junto.

“Defina BENTO_DB_SENHA no arquivo .env”

O Docker Compose não encontrou o `.env`. Rode `iniciar.cmd` ou `iniciar.sh`, que criam o arquivo, ou copie `.env.example` para `.env` e preencha a senha e o segredo.

O npm reclama da versão do Node.js

O projeto exige Node.js 22 ou mais novo. Confira com `node -v` e atualize pelo site oficial ou pelo gerenciador de versões que você usa.

“A API não respondeu” ao abrir o Vite

Em desenvolvimento, a interface em 5173 depende da API em 8080. Use `npm run dev`, que sobe as duas, em vez de `npx vite` sozinho.

13.2 Dados e acesso

Quero apagar tudo e voltar aos dados de demonstração

Com PGlite, pare o servidor e apague a pasta `dados/`. Com Docker, rode `docker compose down -v` e suba de novo. A semente roda sozinha no banco vazio.

Esqueci a senha do administrador

Outro administrador pode usar **Redefinir senha** na gaveta do usuário. Se não houver outro, defina `BENTO_ADMIN_EMAIL`, `BENTO_ADMIN_SENHA` e `BENTO_ADMIN_NOME` com um e-mail novo, inicie o servidor, entre com essa conta e redefina a senha da antiga.

“Muitas tentativas sem sucesso”

Depois de cinco senhas erradas seguidas, o login daquela conta fica suspenso por 15 minutos. Espere o prazo ou peça a um administrador para redefinir a senha.

Todo mundo saiu da conta de repente

O segredo das sessões mudou. Isso acontece quando `BENTO_SEGREDO_JWT` é trocado ou quando a pasta `dados/` foi apagada sem o segredo estar definido no `.env`. Defina um segredo fixo para evitar.

Copyright (©) 2026 Erik Pimenta

13.3 Tempo real e integrações

O indicador fica em “Reconectando” atrás de um proxy

O proxy está segurando a resposta. Desligue o buffer na rota `/api/v1/notificacoes/fluxo` (`proxy_buffering off` no Nginx, `flush_interval -1` no Caddy) e aumente o tempo limite de leitura para mais de 30 segundos.

O teste do webhook falha

A URL precisa ser acessível a partir do servidor do Bento e responder em até 5 segundos com status 2xx. A mensagem de erro aparece no cartão da integração; com `BENTO_LOG=debug`, o log mostra os detalhes.

Um gráfico ficou com as cores erradas depois de mudar a paleta

Os gráficos leem as cores das variáveis CSS ao desenhar. Se você criou uma paleta, confira se ela declara os onze tons de `primaria`, `destaque` e `neutra`.

Copyright (©) 2026 Erik Pimenta

14 Licenças

O Bento é distribuído sob a licença MIT. Os componentes de terceiros mantêm as próprias licenças, todas permissivas.

14.1 Componentes de terceiros

Componente	Uso no projeto	Licença
Tailwind CSS	Estilos e componentes	MIT
Vite	Servidor de desenvolvimento e build	MIT
ApexCharts 4.7.0	Gráficos interativos	MIT
Lucide	Ícones	ISC
Instrument Sans, Bricolage Grotesque, IBM Plex Mono (via Fontsource)	Tipografia	OFL-1.1
Fastify e plugins oficiais	Servidor da API	MIT
Zod	Validação	MIT
node-postgres (pg)	Driver do PostgreSQL	MIT
PGlite	PostgreSQL embutido	Apache-2.0
bcrypt.js	Hash de senhas	BSD-3-Clause
PDFKit	Relatórios em PDF	MIT
TypeScript, tsx	Linguagem e execução em desenvolvimento	Apache-2.0, MIT

> **ApexCharts**
> O ApexCharts está fixado na versão 4.7.0, a última publicada sob a licença MIT. As versões a partir da 5.1 adotaram uma licença dupla que exige compra para empresas com receita anual acima de US$ 2 milhões. Confira a licença antes de atualizar.

O PostgreSQL roda como serviço externo pelo Docker e não é redistribuído no repositório. Slack, Discord, Google Analytics e Stripe são marcas dos seus donos e aparecem apenas como destinos de integração.

14.2 Licença do Bento

Copyright (©) 2026 Erik Pimenta

O código é distribuído sob a licença MIT: você pode usar, copiar, modificar, mesclar, publicar, distribuir, sublicenciar e vender cópias, inclusive para fins comerciais, desde que mantenha o aviso de copyright e o texto da licença em todas as cópias ou partes substanciais. O software é fornecido sem garantia de qualquer tipo. O texto completo está no arquivo `LICENSE`.